

Sipser, *Introduction to the Theory of Computation* (3E)

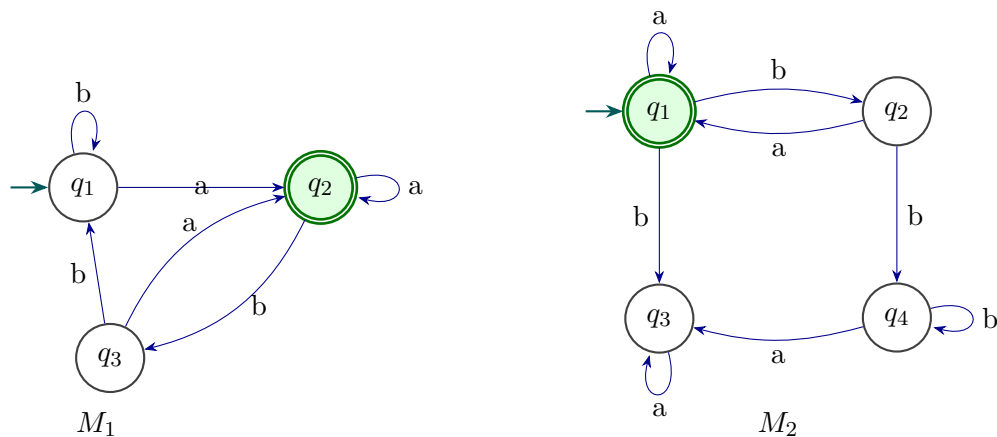
Chapter 1 — Regular Languages

Exercises 1.1–1.30 and Problems 1.31–1.32, with Solutions

Note on figures reproduced from the text: Exercises 1.1, 1.16, 1.21, and 1.24 give pre-existing state diagrams as part of the problem. These have been redrawn here as faithfully as possible from the printed diagrams, cross-checked against the requirement that every DFA transition function be complete and deterministic. If any label differs from your printed copy, the *method* shown for each part still applies directly to the correct diagram.

Exercise 1.1

Problem. The following are the state diagrams of two DFAs, M_1 and M_2 . Answer the following questions about each of these machines.



- What is the start state?
- What is the set of accept states?
- What sequence of states does the machine go through on input **aabb**?
- Does the machine accept the string **aabb**?
- Does the machine accept the string ε ?

Solution. Reading off each diagram (start state = incoming unlabeled arrow; accept states = double circles):

M_1

Transition function read from the diagram: $\delta(q_1, a) = q_2$, $\delta(q_1, b) = q_1$, $\delta(q_2, b) = q_3$, $\delta(q_3, a) = q_2$, $\delta(q_3, b) = q_1$ (and $\delta(q_2, a) = q_2$, completing q_2 's transitions).

a. Start state: q_1 .

b. Accept states: $\{q_2\}$.

c. On **aabb**: $q_1 \xrightarrow{a} q_2 \xrightarrow{a} q_2 \xrightarrow{b} q_3 \xrightarrow{b} q_1$. Sequence: q_1, q_2, q_2, q_3, q_1 .

d. Ends in $q_1 \notin \{q_2\}$: **does not accept aabb**.

e. On ε , the machine halts immediately in $q_1 \notin \{q_2\}$: **does not accept ε** .

M_2

Transition function read from the diagram: $\delta(q_1, a) = q_1$, $\delta(q_1, b) = q_3$, $\delta(q_2, a) = q_1$, $\delta(q_2, b) = q_4$, $\delta(q_3, a) = q_3$, $\delta(q_3, b) = q_2$, $\delta(q_4, a) = q_3$, $\delta(q_4, b) = q_4$.

a. Start state: q_1 .

b. Accept states: $\{q_1\}$.

c. On **aabb**: $q_1 \xrightarrow{a} q_1 \xrightarrow{a} q_1 \xrightarrow{b} q_3 \xrightarrow{b} q_2$. Sequence: q_1, q_1, q_1, q_3, q_2 .

d. Ends in $q_2 \notin \{q_1\}$: **does not accept aabb**.

e. On ε , the machine halts immediately in $q_1 \in \{q_1\}$: **accepts ε** .

Exercise 1.2

Problem. Give the formal description of the machines M_1 and M_2 pictured in Exercise 1.1.

Solution.

$$M_1 = (\{q_1, q_2, q_3\}, \{a, b\}, \delta_1, q_1, \{q_2\}),$$

where δ_1 is given by

	a	b
q_1	q_2	q_1
q_2	q_2	q_3
q_3	q_2	q_1

$$M_2 = (\{q_1, q_2, q_3, q_4\}, \{a, b\}, \delta_2, q_1, \{q_1\}),$$

where δ_2 is given by

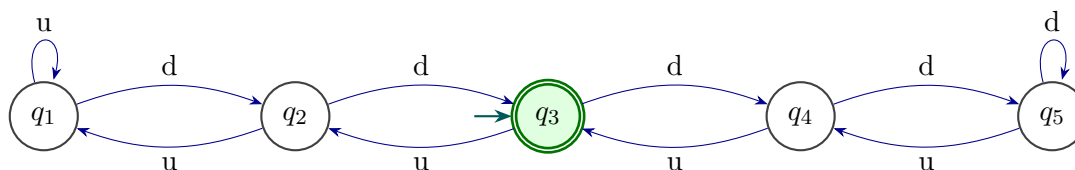
	a	b
q_1	q_1	q_3
q_2	q_1	q_4
q_3	q_3	q_2
q_4	q_3	q_4

Exercise 1.3

Problem. The formal description of a DFA M is $(\{q_1, q_2, q_3, q_4, q_5\}, \{u, d\}, \delta, q_3, \{q_3\})$, where δ is given by the following table. Give the state diagram of this machine.

	u	d
q_1	q_1	q_2
q_2	q_1	q_3
q_3	q_2	q_4
q_4	q_3	q_5
q_5	q_4	q_5

Solution. This machine tracks a counter (think of u =“up”, d =“down”) ranging over $q_1, \dots, q_5$, incremented by d and decremented by u , staying put at the ends.



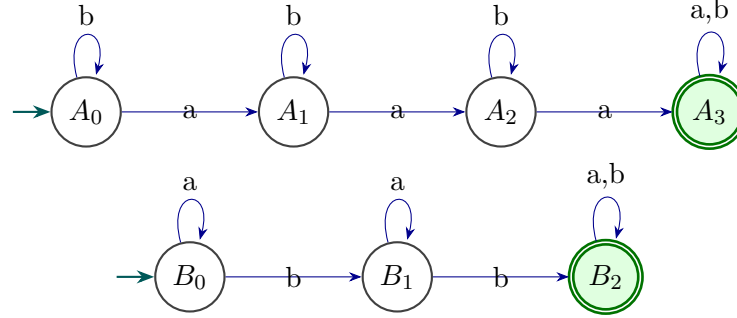
Exercise 1.4

Problem. Each of the following languages is the intersection of two simpler languages. In each part, construct DFAs for the simpler languages, then combine them using the product (cross-product) construction to give the state diagram of a DFA for the language given. In all parts, $\Sigma = \{a, b\}$.

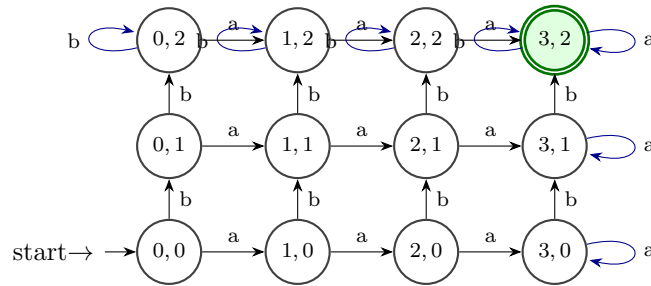
- $\{w \mid w \text{ has at least three a's and at least two b's}\}$
- $\{w \mid w \text{ has exactly two a's and at least two b's}\}$
- $\{w \mid w \text{ has an even number of a's and one or two b's}\}$
- $\{w \mid w \text{ has an even number of a's and each a is followed by at least one b}\}$
- $\{w \mid w \text{ starts with an a and has at most one b}\}$
- $\{w \mid w \text{ has an odd number of a's and ends with a b}\}$
- $\{w \mid w \text{ has even length and an odd number of a's}\}$

Solution. For each part we build the two component DFAs, then the product DFA whose states are pairs (i, j) with one component from each machine; an a -edge moves within a row (changes the first coordinate), a b -edge moves within a column (changes the second coordinate). Only states satisfying *both* original accepting conditions are accepting in the product.

(a) at least three a's, and at least two b's

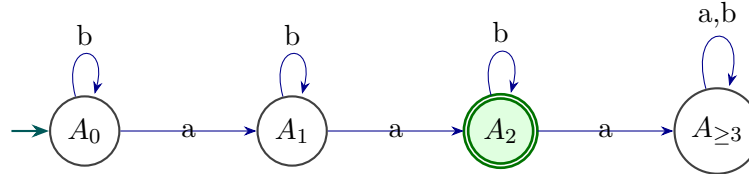


Product DFA (only (A_3, B_2) is accepting):

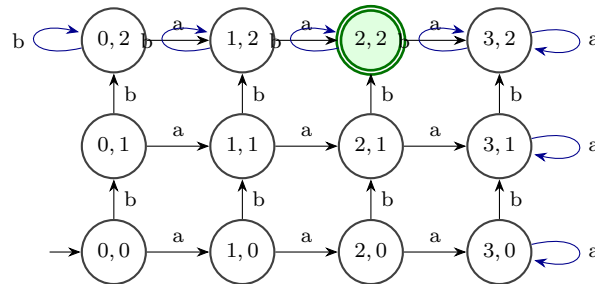


(b) exactly two a's, and at least two b's

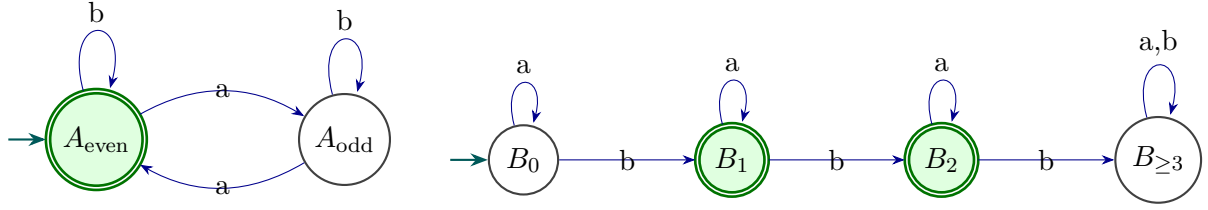
The first component now caps at “exactly 2”: states 0, 1, 2(accept), ≥ 3 (trap), incrementing on a , capped/trapped past 2.



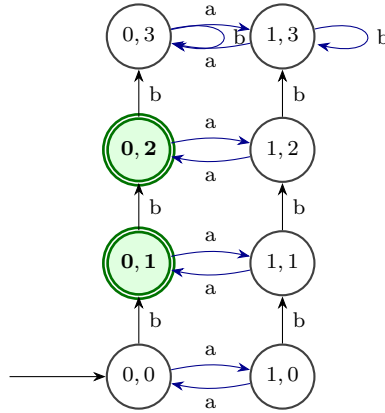
The second component is the same B_0, B_1, B_2 (“at least two b's”) machine as in part (a). The product has $4 \times 3 = 12$ states, grid-connected exactly as in part (a), with the single accepting state (A_2, B_2) .



(c) even number of a's, and one or two b's

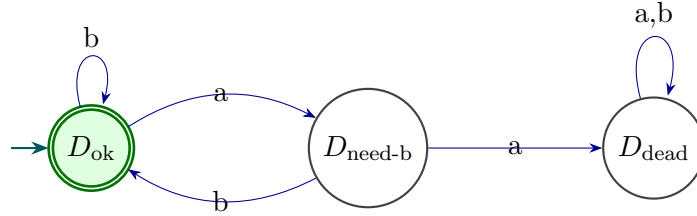


Product ($2 \times 4 = 8$ states; row index $i \in \{0=\text{even}, 1=\text{odd}\}$ tracks a -parity, column index $j \in \{0, 1, 2, 3=\text{trap}\}$ tracks the b -count; accepting states are $(0, 1)$ and $(0, 2)$, shown bold):

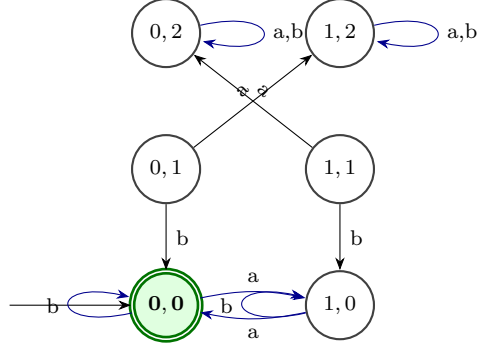


(d) even number of a's, and each a is followed by at least one b

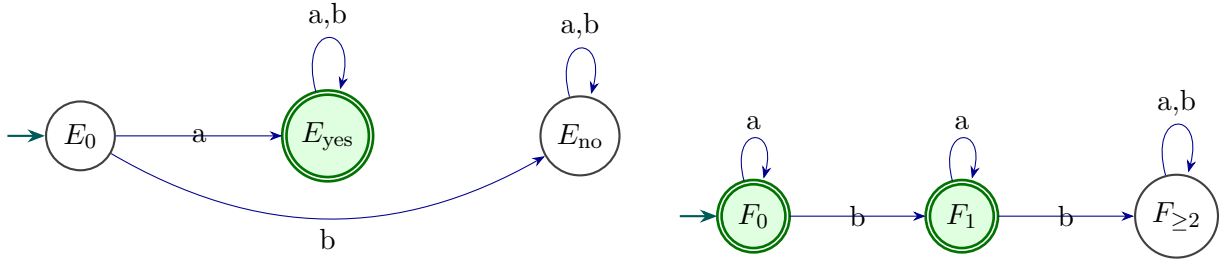
Second component (“no aa, and does not end in a”): D_{ok} (start/just-saw-b, accepting), $D_{\text{need-b}}$ (just saw an a), D_{dead} (saw aa).



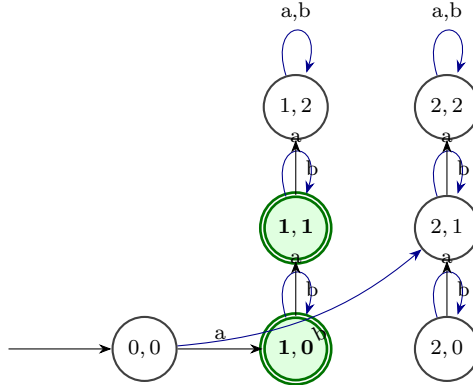
Combined with the even/odd- a -count machine $A_{\text{even}}, A_{\text{odd}}$ from part (c), the product has $2 \times 3 = 6$ states; row index $i \in \{0=\text{even}, 1=\text{odd}\}$, column index $j \in \{0=\text{ok}, 1=\text{need-b}, 2=\text{dead}\}$. Note $\delta((i, 1), a) = (\bar{i}, 2)$ (an a from “need-b” both flips the a -parity *and* goes to dead — drawn as the diagonal edges below). The unique accepting state is $(0, 0)$.



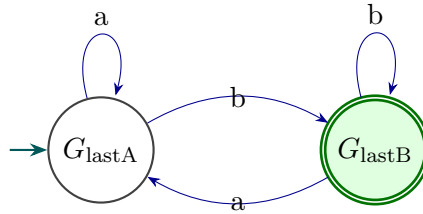
(e) starts with an a, and has at most one b



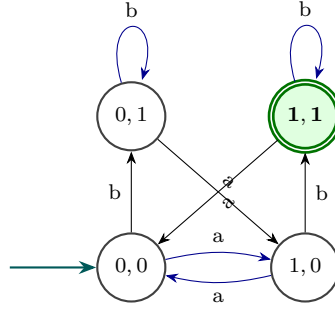
Product ($3 \times 3 = 9$ states; row $i \in \{0=E_0, 1=E_{\text{yes}}, 2=E_{\text{no}}\}$, column $j \in \{0, 1, 2=\geq 2\}$ counts b 's; accepting $= \{1\} \times \{0, 1\}$, shown bold). The states $(0, 1), (0, 2)$ are unreachable from the start and are omitted.



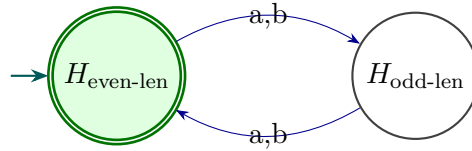
(f) odd number of a's, and ends with a b



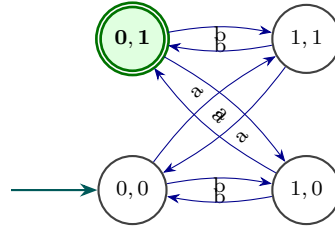
Combined with $A_{\text{even}}, A_{\text{odd}}$ from part (c): product has $2 \times 2 = 4$ states, (i, j) with $i \in \{0=\text{even}, 1=\text{odd}\}$, $j \in \{0=\text{lastA}, 1=\text{lastB}\}$; accepting $= (1, 1)$.



(g) even length, and odd number of a's



Combined with A_{even} , A_{odd} (part (c)): product has $2 \times 2 = 4$ states, (i, j) with $i \in \{0=\text{even-len}, 1=\text{odd-len}\}$, $j \in \{0=\text{even-a}, 1=\text{odd-a}\}$; accepting = $(0, 1)$.



Here $(i, j) \xrightarrow{a} (1-i, 1-j)$ (an a flips both length-parity and a -count-parity) while $(i, j) \xrightarrow{b} (1-i, j)$ (a b flips only length-parity) — giving the two crossing diagonals (both via a) and the two horizontal edges (both via b) shown.

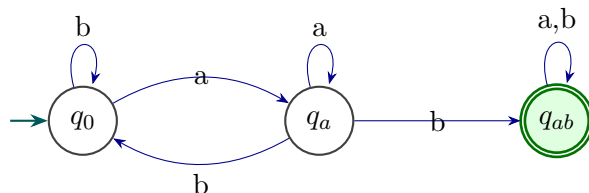
Exercise 1.5

Problem. Each of the following languages is the complement of a simpler language. In each part, construct a DFA for the simpler language, then use it to give the state diagram of a DFA for the language given. In all parts, $\Sigma = \{a, b\}$.

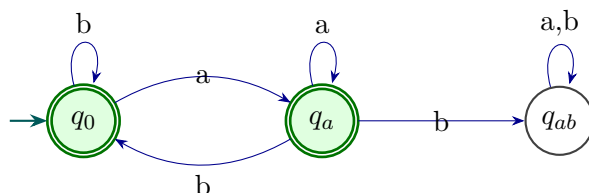
- $\{w \mid w \text{ does not contain the substring } ab\}$
- $\{w \mid w \text{ does not contain the substring } baba\}$
- $\{w \mid w \text{ contains neither the substrings } ab \text{ nor } ba\}$
- $\{w \mid w \text{ is any string not in } a^*b^*\}$
- $\{w \mid w \text{ is any string not in } (ab^+)^*\}$
- $\{w \mid w \text{ is any string not in } a^* \cup b^*\}$
- $\{w \mid w \text{ is any string that doesn't contain exactly two a's}\}$
- $\{w \mid w \text{ is any string except } a \text{ and } b\}$

Solution. In each part: build a DFA for the *simpler* language first; the complement DFA has *exactly the same states and transitions*, with the accepting set replaced by its complement within Q (Exercise 1.14a justifies this swap).

(a) complement of “contains ab”

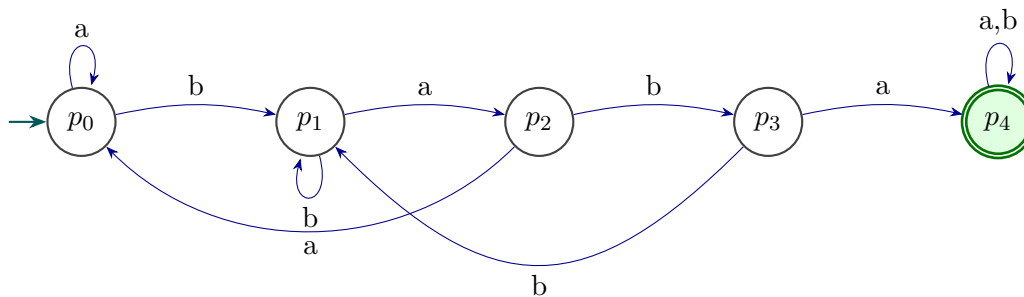


Simpler language “contains **ab**” is accepted by $\{q_{ab}\}$. **Complement** (does not contain **ab**): same diagram, accept set = $\{q_0, q_a\}$ (both circled double below).



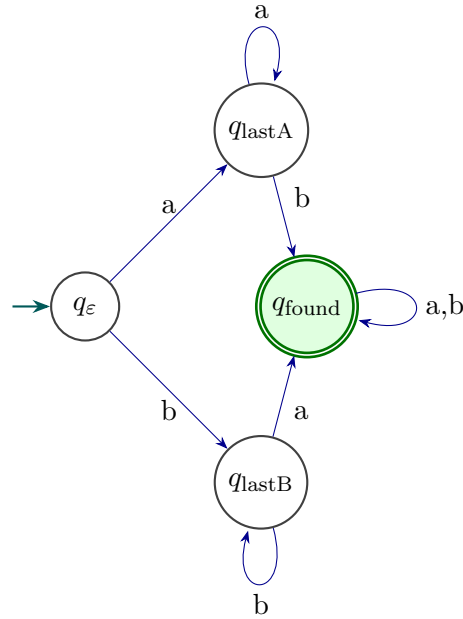
(b) complement of “contains baba”

Standard substring-matching (KMP-style) automaton tracking the longest prefix of **baba** matched by the input’s suffix:



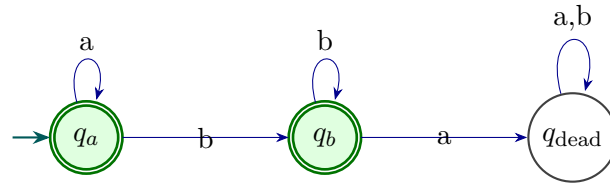
Accept set for “contains **baba**” is $\{p_4\}$; for the **complement** (does not contain **baba**), accept set = $\{p_0, p_1, p_2, p_3\}$ (same diagram, all states except p_4 double-circled).

(c) complement of “contains ab or ba”



Accept set for “contains **ab** or **ba**” is $\{q_{\text{found}}\}$. **Complement** (neither substring occurs): accept set = $\{q_\epsilon, q_{\text{lastA}}, q_{\text{lastB}}\}$. (Note: reaching q_{lastA} or q_{lastB} and staying there forever means the whole string is a run of a single repeated letter, e.g. **aaaa** or **bbb** or ϵ — exactly the strings with neither substring.)

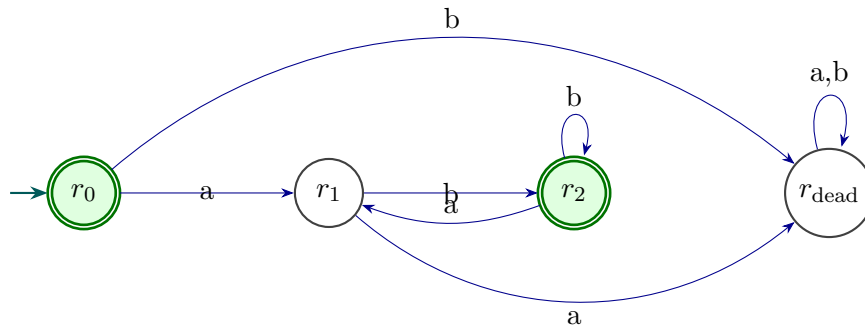
(d) complement of a^*b^*



Accept set for a^*b^* is $\{q_a, q_b\}$. **Complement**: accept set = $\{q_{\text{dead}}\}$ (a string is outside a^*b^* exactly when some **b** is eventually followed by an **a**).

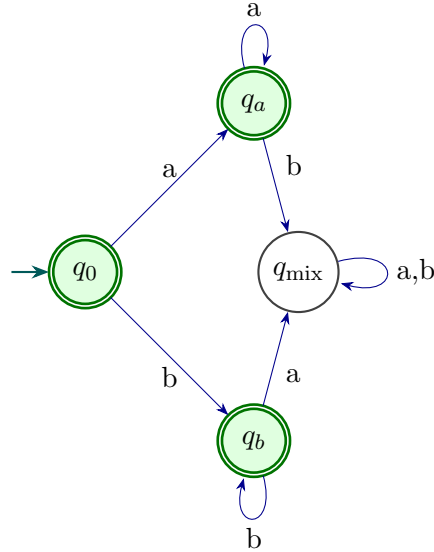
(e) complement of $(ab^+)^*$

$(ab^+)^*$: zero or more blocks, each block one *a* followed by *one or more b*'s.



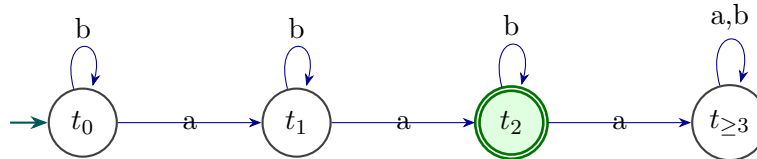
Accept set for $(ab^+)^*$ is $\{r_0, r_2\}$ (a completed sequence of blocks). **Complement:** accept set $= \{r_1, r_{\text{dead}}\}$.

(f) complement of $a^* \cup b^*$



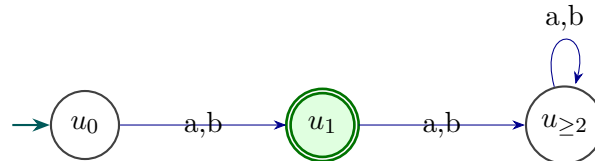
Accept set for $a^* \cup b^*$ is $\{q_0, q_a, q_b\}$. **Complement:** accept set $= \{q_{\text{mix}}\}$ (the string contains *both* an a and a b).

(g) complement of “exactly two a’s”



Accept set for “exactly two a’s” is $\{t_2\}$. **Complement** (not exactly two a’s): accept set $= \{t_0, t_1, t_{\geq 3}\}$.

(h) complement of $\{a,b\}$



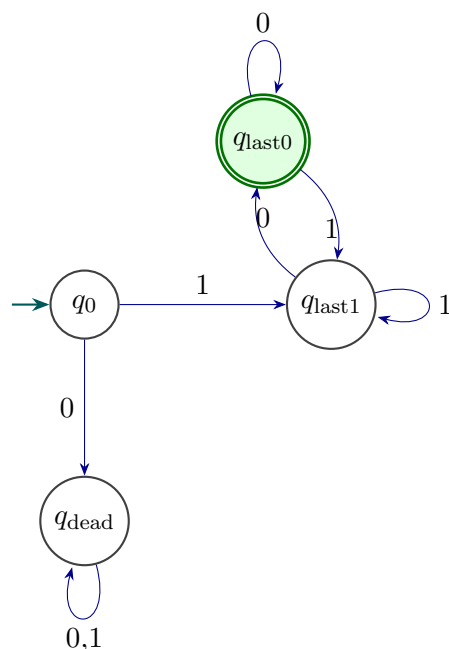
Accept set for $\{a, b\}$ is $\{u_1\}$ (exactly one symbol read). **Complement** (any string except a and b , including ϵ and all strings of length ≥ 2): accept set $= \{u_0, u_{\geq 2}\}$.

Exercise 1.6

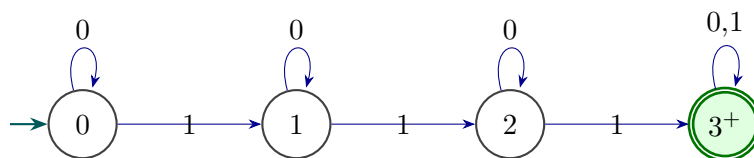
Problem. Give state diagrams of DFAs recognizing the following languages. In all parts, the alphabet is $\{0,1\}$.

- $\{w \mid w \text{ begins with a 1 and ends with a 0}\}$
- $\{w \mid w \text{ contains at least three 1s}\}$
- $\{w \mid w \text{ contains the substring } 0101\}$
- $\{w \mid w \text{ has length at least 3 and its third symbol is a 0}\}$
- $\{w \mid w \text{ starts with 0 and has odd length, or starts with 1 and has even length}\}$
- $\{w \mid w \text{ doesn't contain the substring } 110\}$
- $\{w \mid \text{the length of } w \text{ is at most 5}\}$
- $\{w \mid w \text{ is any string except } 11 \text{ and } 111\}$
- $\{w \mid \text{every odd position of } w \text{ is a 1}\}$
- $\{w \mid w \text{ contains at least two 0s and at most one 1}\}$
- $\{\varepsilon, 0\}$
- $\{w \mid w \text{ contains an even number of 0s, or contains exactly two 1s}\}$
- the empty set
- all strings except the empty string

(a) begins with 1, ends with 0

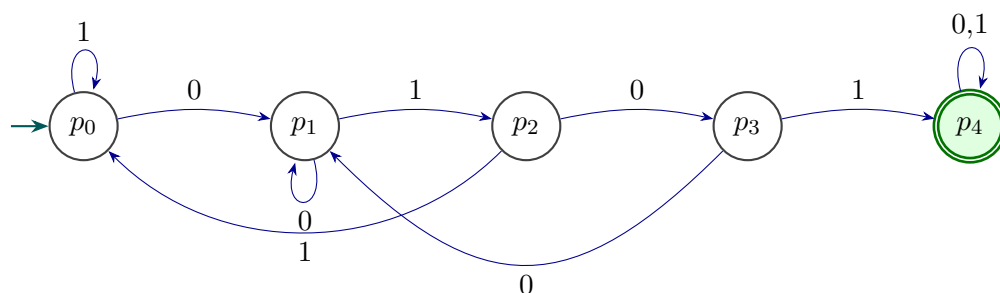


(b) at least three 1s

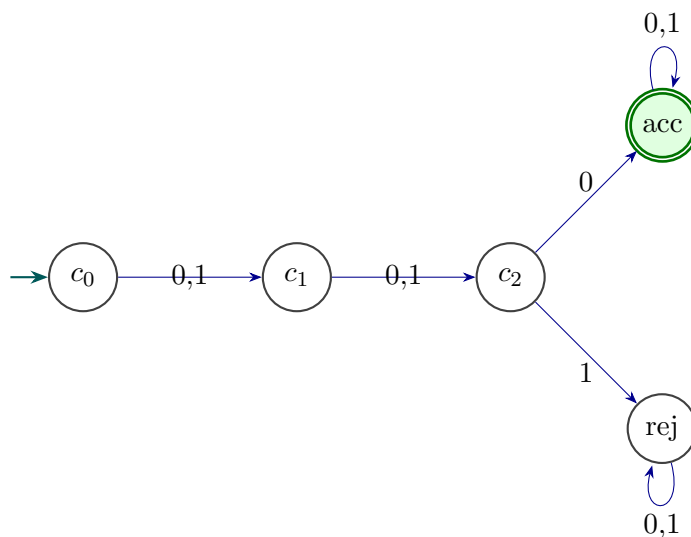


(c) contains substring 0101

KMP automaton for pattern 0101:

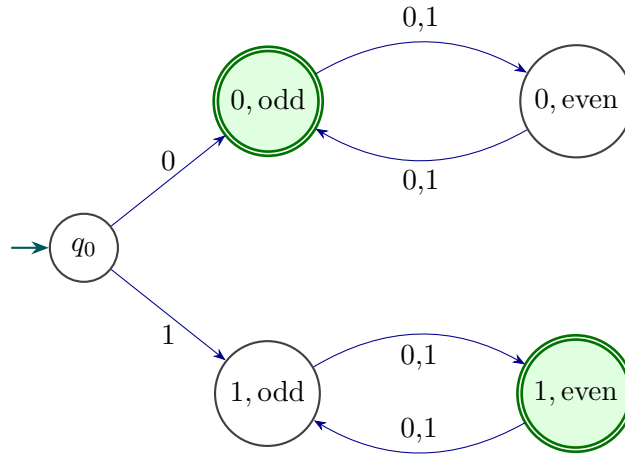


(d) length ≥ 3 , third symbol is 0

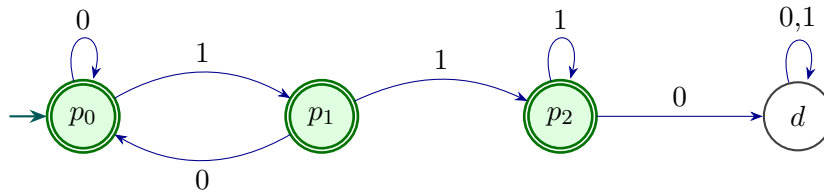


(e) (starts 0, odd length) or (starts 1, even length)

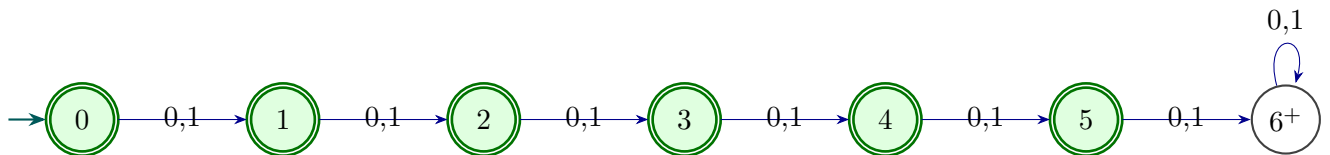
State (x, p) : x = first symbol, p = current length parity; accept iff $(x=0, p=\text{odd})$ or $(x=1, p=\text{even})$.



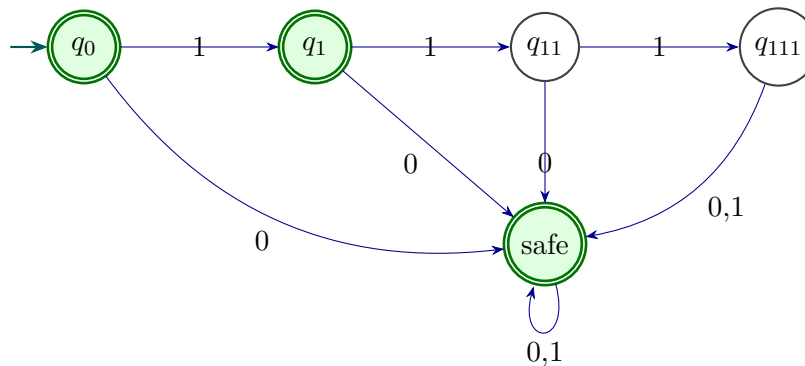
(f) does not contain substring 110



(g) length at most 5

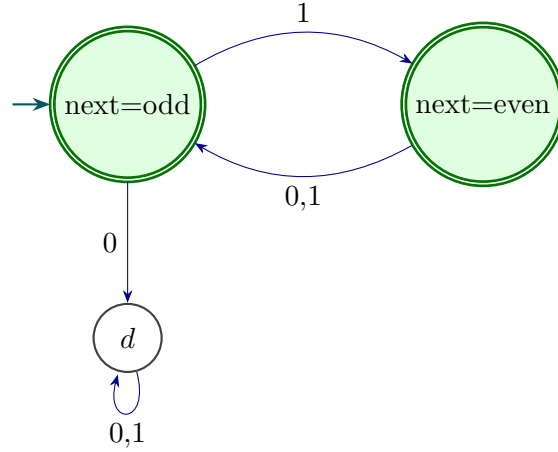


(h) any string except 11 and 111



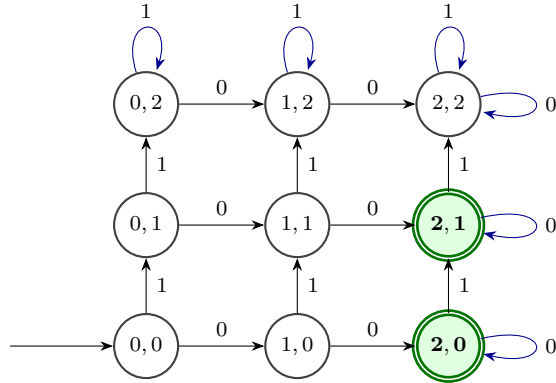
(i) every odd position is a 1

State names the parity of the *next* position to be read.

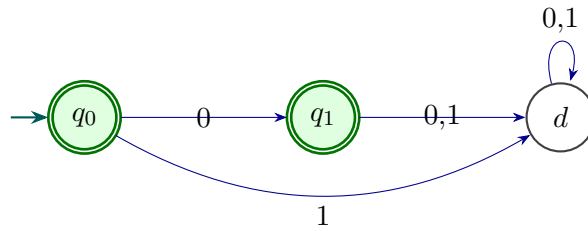


(j) at least two 0s and at most one 1

Grid (i, j) : $i \in \{0, 1, 2\}$ counts 0s (capped), $j \in \{0, 1, 2\}$ counts 1s (capped); accept = $\{(2, 0), (2, 1)\}$.

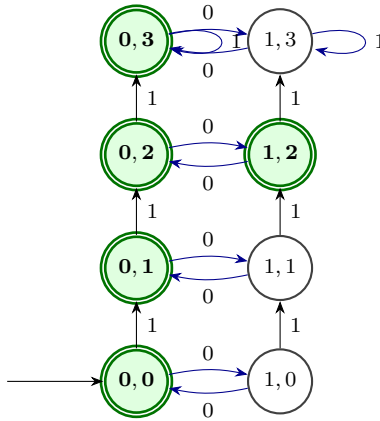


(k) $\{\varepsilon, 0\}$



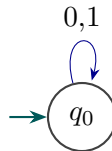
(l) even number of 0s, or exactly two 1s

Grid (i, j) : $i \in \{\text{even}, \text{odd}\}$ (0-parity), $j \in \{0, 1, 2, 3^+\}$ (1-count capped); accept = $\{i=\text{even}\} \cup \{j=2\}$ (bold).



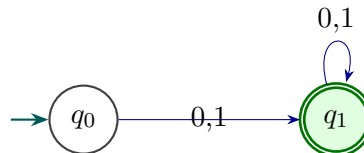
(Here “0” means the symbol 0, tracked by the row toggle; “1” means the symbol 1, tracked by the column count — unrelated to the digits used as state-index labels.)

(m) the empty set $\emptyset$



No accept states at all — no string is ever accepted.

(n) all strings except ε

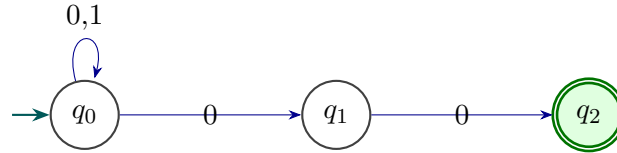


Exercise 1.7

Problem. Give state diagrams of NFAs with the specified number of states recognizing each of the following languages. In all parts, the alphabet is $\{0, 1\}$.

- $\{w \mid w \text{ ends with } 00\}$ with three states
- the language of Exercise 1.6c with five states
- the language of Exercise 1.6l with six states
- $\{0\}$ with two states
- $0^*1^*0^+$ with three states
- $1^*(001^+)^*$ with three states
- $\{\varepsilon\}$ with one state
- 0^* with one state

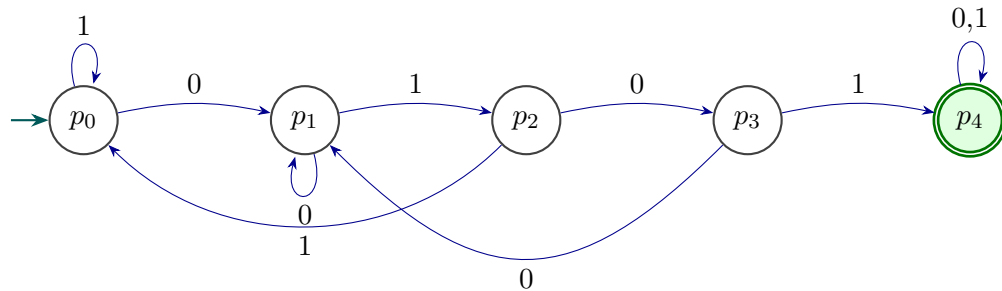
(a) ends with 00 — 3 states



Nondeterministically “guesses” where the final 00 begins.

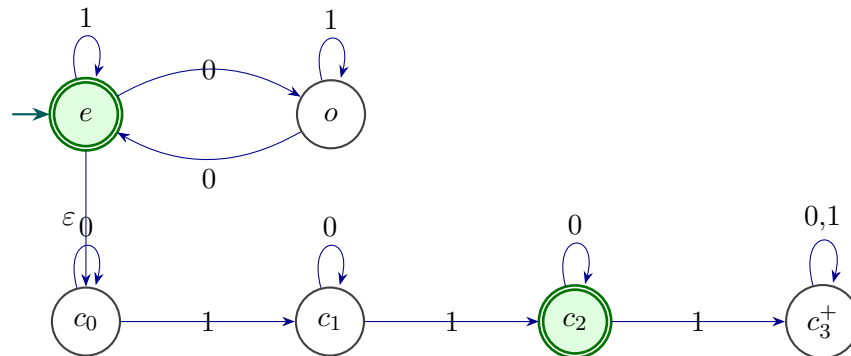
(b) Exercise 1.6c (contains 0101) — 5 states

The DFA already built for 1.6c has exactly 5 states (every DFA is itself an NFA), so we reuse it unchanged.



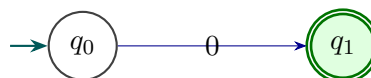
(c) Exercise 1.6l (even 0s, or exactly two 1s) — 6 states

Run the two conditions in parallel via nondeterminism, merging the shared start state with the “even-0s” branch’s start (which is already accepting, since 0 zeros is even), and branching by ε into the “exactly two 1s” branch.



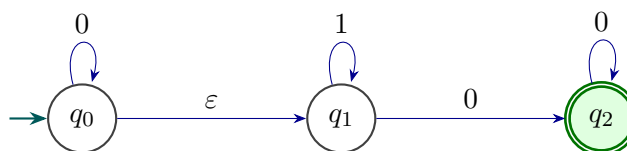
Reading a string, the NFA can nondeterministically stay in the $\{e, o\}$ branch (accepting iff it ends on e , i.e. even 0-count) or take the ε -move into the $\{c_0, \dots, c_3^+\}$ branch (accepting iff it ends on c_2 , i.e. exactly two 1s); the machine accepts iff *some* branch accepts, exactly realizing the union.

(d) $\{0\}$ — 2 states

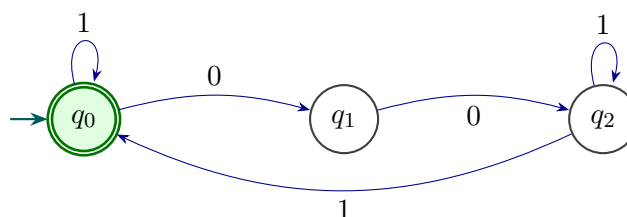


No transition is defined on 1 from q_0 , nor on anything from q_1 : any other string simply has no accepting path, hence is rejected.

(e) $0^*1^*0^+$ — 3 states



(f) $1^*(001^+)^*$ — 3 states



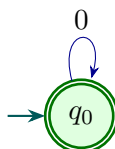
From q_2 , reading a 1 nondeterministically either stays at q_2 (absorbing further 1s within the current 001+ block) or jumps back to q_0 (declaring the block complete); both are valid NFA transitions on the same symbol.

(g) $\{\varepsilon\}$ — 1 state



No transitions at all: any nonempty input has no move to follow, so it is rejected; only ε (which reads nothing) is accepted, since q_0 is itself accepting.

(h) 0^* — 1 state



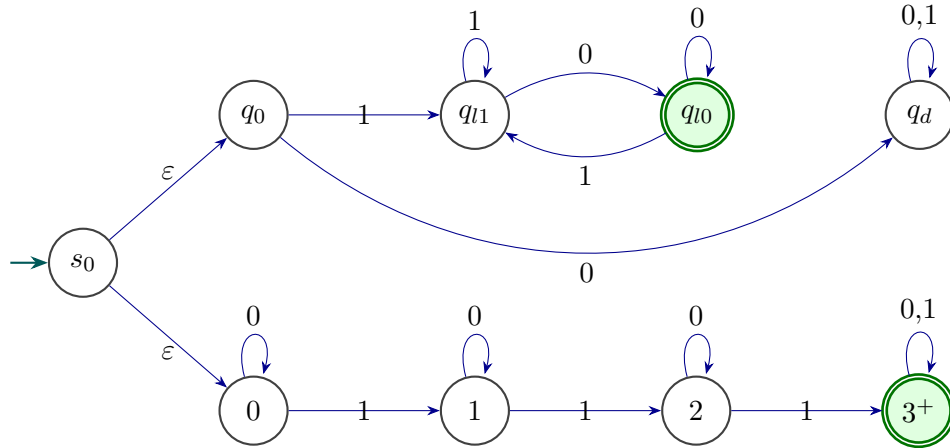
Exercise 1.8

Problem. Use the construction in the proof of Theorem 1.45 (union) to give the state diagrams of NFAs recognizing the union of the languages described in

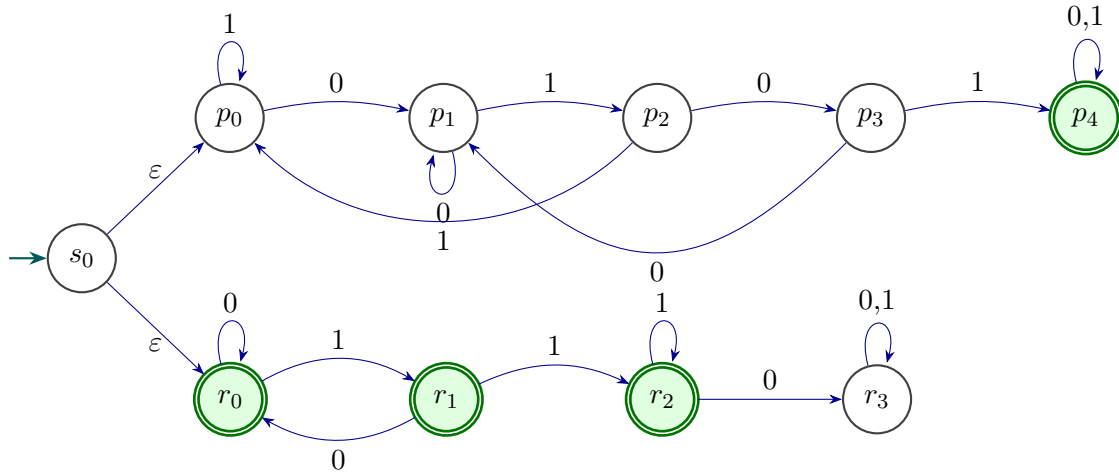
- Exercises 1.6a and 1.6b.
- Exercises 1.6c and 1.6f.

Solution. The construction adds a new start state with ε -transitions to each machine's old start state; the accept states are simply the union of both machines' accept states.

(a) (begins 1, ends 0) $\cup$ (at least three 1s)



(b) (contains 0101) $\cup$ (does not contain 110)



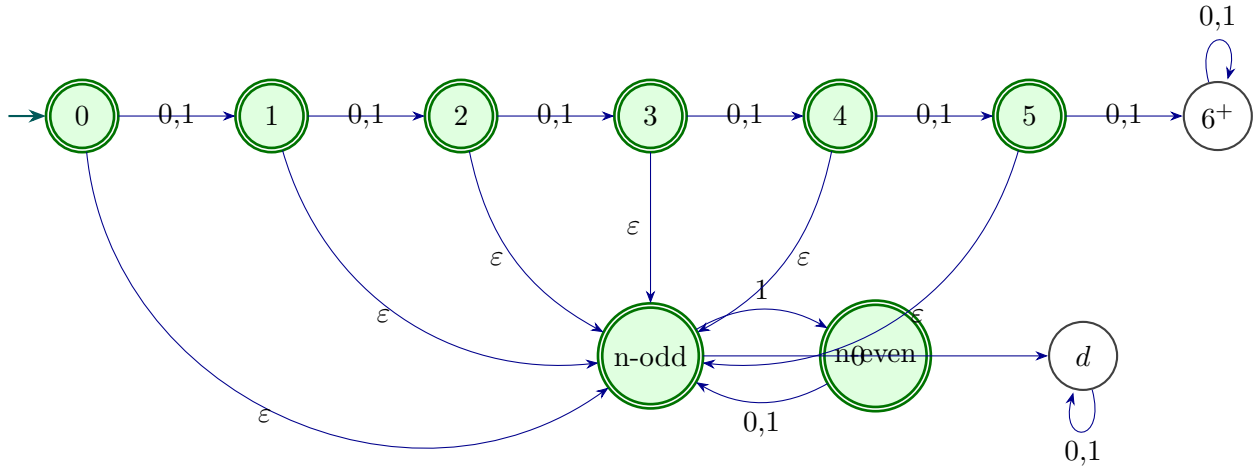
Exercise 1.9

Problem. Use the construction in the proof of Theorem 1.47 (concatenation) to give the state diagrams of NFAs recognizing the concatenation of the languages described in

- Exercises 1.6g and 1.6i.
- Exercises 1.6b and 1.6m.

Solution. The construction adds an ε -transition from every accept state of N_1 to the start state of N_2 ; the accept states of the combined machine are exactly those of N_2 .

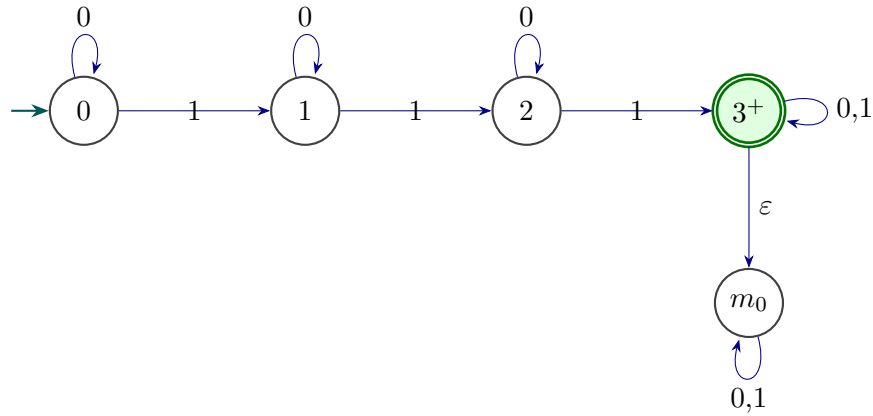
(a) $(\text{length} \leq 5) \cdot (\text{every odd position is } 1)$



(Accept states of the combined machine: $\{\text{n-odd}, \text{n-even}\}$, the accept states of the second component only. Note l_0 stays accepting because it can ε -reach the accepting start of the second machine, correctly reflecting that $\varepsilon \cdot \varepsilon = \varepsilon$ is in the concatenation.)

(b) $(\text{at least three } 1\text{s}) \cdot \emptyset$

Since $L(1.6\text{m}) = \emptyset$, and $A \cdot \emptyset = \emptyset$ for every language A (there is no string in $\emptyset$ to append), the construction below has an ε -edge into 1.6m's single, non-accepting, dead-end state — and hence accepts nothing, correctly realizing $\emptyset$.



The combined machine's accept states are those of 1.6m — which is $\emptyset$ (no accept states at all), so the whole NFA has *no* accept states and recognizes $\emptyset$.

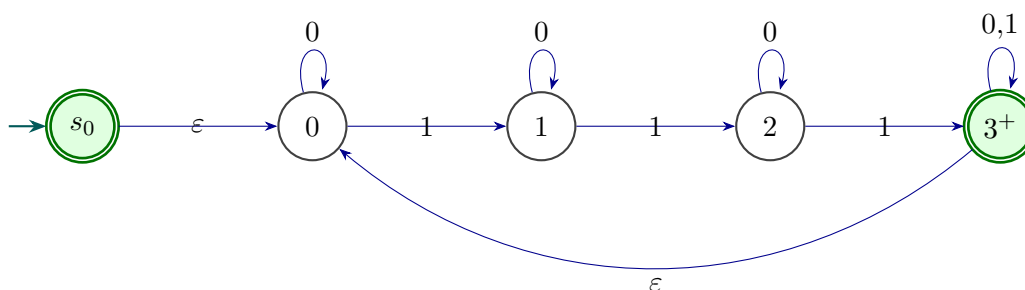
Exercise 1.10

Problem. Use the construction in the proof of Theorem 1.49 (star) to give the state diagrams of NFAs recognizing the star of the languages described in

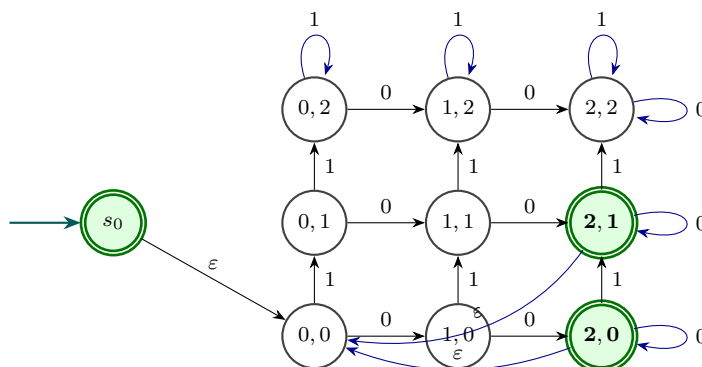
- a. Exercise 1.6b.
- b. Exercise 1.6j.
- c. Exercise 1.6m.

Solution. The construction adds a new start state, which is also accepting (to accept ε), with an ε -edge to the old start state; every old accept state gets an added ε -edge back to the old start state.

(a) (at least three 1s)*

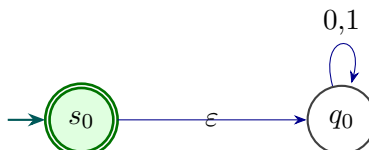


(b) (at least two 0s and at most one 1)*



(c) $\emptyset^*$

Since 1.6m's single state q_0 has *no* accept states, the construction adds no “loop back” ε -edges at all — only the new accepting start s_0 with its ε -edge to q_0 :

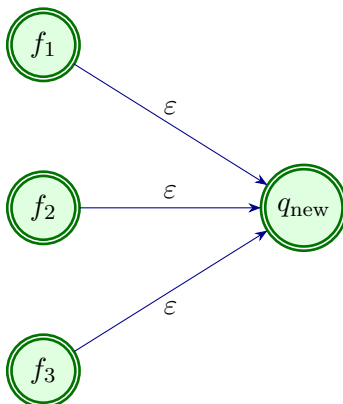


This machine accepts exactly ε (via the accepting start state) and rejects everything else, correctly realizing the identity $\emptyset^* = \{\varepsilon\}$.

Exercise 1.11

Problem. Prove that every NFA can be converted to an equivalent one that has a single accept state.

Solution. Let $N = (Q, \Sigma, \delta, q_0, F)$ be an NFA. If $|F| \leq 1$ we are done. Otherwise construct $N' = (Q \cup \{q_{\text{new}}\}, \Sigma, \delta', q_0, \{q_{\text{new}}\})$ where q_{new} is a new state not in Q , and δ' equals δ except that every state in F additionally gets an ε -transition to q_{new} (and no state in Q is accepting in N' ; only q_{new} is).



Correctness. A string $w = w_1 \cdots w_k$ is accepted by N iff some accepting run $q_0, r_1, \dots, r_k$ exists with $r_k \in F$. This exact run is also a valid run of N' on w (since $\delta' \supseteq \delta$ on Q), and it can be extended by one free ε -move from $r_k \in F$ to q_{new} , giving an accepting run of N' . Conversely, any accepting run of N' ends at q_{new} , which is only reachable by an ε -move from some $f \in F$ after processing all of w within Q — so that prefix run is a valid accepting run of N . Hence $L(N') = L(N)$, and N' has exactly one accept state. ■

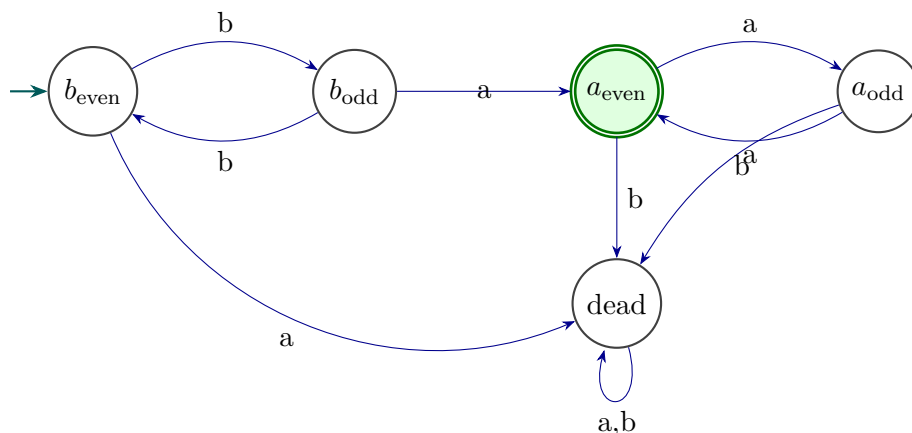
Exercise 1.12

Problem. Let $D = \{w \mid w \text{ contains an even number of a's and an odd number of b's and does not contain the substring } ab\}$. Give a DFA with five states that recognizes D and a regular expression that generates D . (Suggestion: describe D more simply.)

Solution. Simpler description. If w contains no **ab** substring, then once an a appears, no b can appear afterward (that would create **ab**). So every string with no **ab** substring has the form $b^m a^n$ — some (possibly zero) b's, followed by some (possibly zero) a's. Requiring an even number of a's and odd number of b's: $D = \{b^m a^n \mid m \text{ odd}, n \text{ even}\}$.

Regular expression. $D = b(bb)^*(aa)^*$.

DFA (5 states).

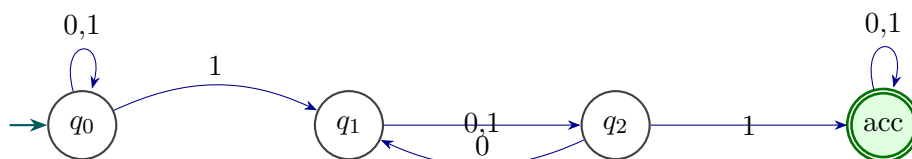


Exercise 1.13

Problem. Let F be the language of all strings over $\{0,1\}$ that do *not* contain a pair of 1s separated by an odd number of symbols. Give the state diagram of a DFA that recognizes F . (You may find it helpful first to find an NFA for the complement of F .)

Solution. Reformulation. For positions $i < j$, the number of symbols strictly between them is $j - i - 1$; this is odd exactly when $j - i$ is even, i.e. exactly when i and j have the *same* parity (both odd-indexed or both even-indexed, 1-indexed). So $\overline{F}$ (the complement) is: *some two 1s occur at positions of the same parity*.

4-state NFA for $\overline{F}$. Nondeterministically guess the first of the pair, then skip an odd number of further symbols before matching the second:

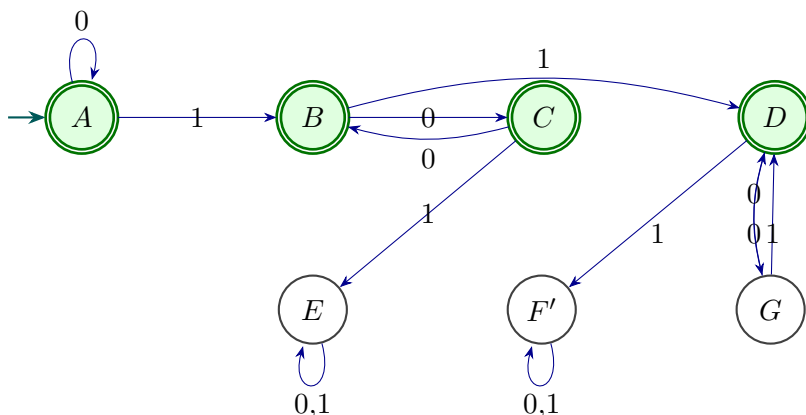


(From q_2 , reading a 1 can nondeterministically either accept, claiming this is the matching second 1, *or* loop back to q_1 to keep searching further along for a later match with the same guessed first 1 — both are legal NFA choices on the same symbol, omitted from the picture as a second edge $q_2 \xrightarrow{1} q_1$ for clarity but formally present.)

Applying the subset construction (Theorem 1.39) to this NFA and tracking reachable subsets of $\{q_0, q_1, q_2, \text{acc}\}$ produces a DFA for $\overline{F}$; swapping accept/non-accept states (Exercise 1.14a) then gives F . Carrying out the subset construction explicitly (state names abbreviate the NFA-subset reached):

	0	1
$\{q_0\}$	$\{q_0\}$	$\{q_0, q_1\}$
$\{q_0, q_1\}$	$\{q_0, q_2\}$	$\{q_0, q_1, q_2\}$
$\{q_0, q_2\}$	$\{q_0, q_1\}$	$\{q_0, q_1, \text{acc}\}$
$\{q_0, q_1, q_2\}$	$\{q_0, q_1, q_2\}$	$\{q_0, q_1, q_2, \text{acc}\}$
$\{q_0, q_1, \text{acc}\}$	$\{q_0, q_2, \text{acc}\}$	$\{q_0, q_1, q_2, \text{acc}\}$
$\{q_0, q_1, q_2, \text{acc}\}$	$\{q_0, q_1, q_2, \text{acc}\}$	$\{q_0, q_1, q_2, \text{acc}\}$
$\{q_0, q_2, \text{acc}\}$	$\{q_0, q_1, \text{acc}\}$	$\{q_0, q_1, \text{acc}\}$

The sets containing acc are accepting for $\overline{F}$ (i.e. *non*-accepting for F). Labeling the seven reachable subsets $A, \dots, G$ in the order listed and swapping the accept sets gives the DFA for F :



(States E and F' are the two non-accepting “already found a violating pair” sinks; both are absorbing.)

Exercise 1.14

Problem.

- Show that if M is a DFA recognizing B , swapping the accept and nonaccept states yields a DFA recognizing the complement of B . Conclude that the class of regular languages is closed under complement.
- Show by giving an example that if M is an NFA recognizing C , swapping the accept and nonaccept states doesn't necessarily yield an NFA recognizing the complement of C . Is the class of languages recognized by NFAs closed under complement?

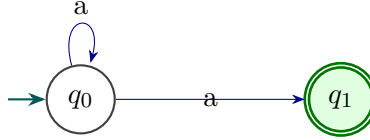
Solution.

(a)

Let $M = (Q, \Sigma, \delta, q_0, F)$ recognize B , and let $M' = (Q, \Sigma, \delta, q_0, Q \setminus F)$ (same states/transitions/start, complemented accept set). Since M is a DFA, on any input w both M and M' follow the *same unique* run $q_0, r_1, \dots, r_n$ (determinism guarantees the run doesn't depend on which states are marked accepting). M accepts w iff $r_n \in F$; M' accepts w iff $r_n \in Q \setminus F$, i.e. iff $r_n \notin F$, i.e. iff M *rejects* w . So $L(M') = \Sigma^* \setminus L(M) = \overline{B}$. Since B was an arbitrary regular language (recognized by some DFA M), and we exhibited a DFA M' recognizing $\overline{B}$, $\overline{B}$ is regular too: the regular languages are closed under complement. ■

(b)

Consider the NFA N with states $\{q_0, q_1\}$, start q_0 , accept $\{q_1\}$, and transitions $\delta(q_0, a) = \{q_0, q_1\}$, $\delta(q_1, a) = \emptyset$, over $\Sigma = \{a\}$.



N accepts exactly $C = \{a^n \mid n \geq 1\} = a^+$ (guess the last step into q_1). Swapping accept states gives N' with accept set $\{q_0\}$. On input a : N' 's runs are $q_0 \rightarrow q_0$ (accepting, since q_0 ends accepting) and $q_0 \rightarrow q_1$ (not accepting) — since *some* run accepts, N' accepts $a \in C$. But $a \in C$ means $a \notin \overline{C}$, so N' does *not* recognize $\overline{C}$ (it wrongly accepts a string that should be rejected). The naive swap fails because an NFA can have *several* runs on the same string, and “some run accepts” does not negate to “some run accepts” under swapping — it negates to “*all* runs reject.”

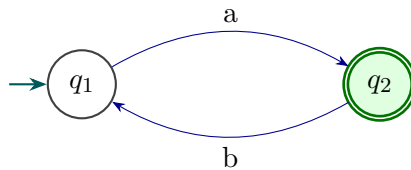
Nonetheless, the class of NFA-recognized languages *is* closed under complement — just not via this naive swap. Since every NFA can be converted to an equivalent DFA (Theorem 1.39), and DFA-recognized languages equal NFA-recognized languages equal the regular languages, closure under complement follows from part (a) applied after first determinizing.

Exercise 1.15

Problem. Give a counterexample to show that the following construction fails to prove Theorem 1.49 (closure of the regular languages under star). Let $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ recognize A_1 . Construct $N = (Q, \Sigma, \delta, q_1, F)$ as follows: $Q = Q_1$; start state is the same; $F = \{q_1\} \cup F_1$; and $\delta(q, a) = \delta_1(q, a)$ if $q \notin F_1$ or $a \neq \varepsilon$, and $\delta(q, a) = \delta_1(q, a) \cup \{q_1\}$ if $q \in F_1$ and $a = \varepsilon$.

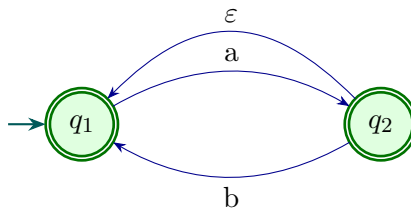
Solution. This construction reuses the *original* start state q_1 as the new accept-for- ε state, instead of adding a genuinely new one. That is exactly what breaks it: if N_1 has some other edge leading back into q_1 (which is perfectly fine in N_1 itself, since incoming edges to the start state are simply never used when running N_1 alone), that edge becomes dangerous once q_1 is *also* marked accepting and targeted by new ε -edges from F_1 — it can let the machine take an illegitimate shortcut.

Counterexample. Let N_1 recognize $A_1 = a(ba)^* = \{a, aba, ababa, \dots\}$ via $Q_1 = \{q_1, q_2\}$, start q_1 (not accepting), $F_1 = \{q_2\}$, $\delta_1(q_1, a) = \{q_2\}$, $\delta_1(q_2, b) = \{q_1\}$:



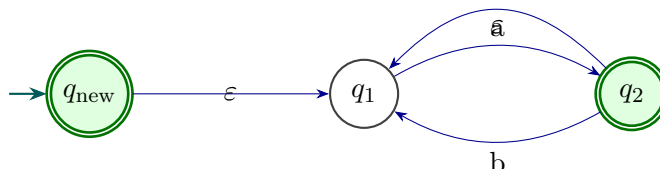
The string ab is *not* in A_1^* : it is not itself in A_1 (all of A_1 's strings have odd length, alternating and starting/ending in a), and its only nontrivial split, $a \cdot b$, has $b \notin A_1$; no split of a 2-letter string into pieces of A_1 works.

Applying the flawed construction: $F = \{q_1, q_2\}$ (so q_1 becomes accepting), and since $q_2 \in F_1$ we add the new transition $\delta(q_2, \varepsilon) \ni q_1$ — *alongside* the pre-existing real edge $\delta_1(q_2, b) = q_1$:



Now trace **ab** on this flawed N : $q_1 \xrightarrow{a} q_2 \xrightarrow{b} q_1$. This run ends at q_1 , which is accepting in the flawed machine (since $F = \{q_1, q_2\}$) — so the flawed N **incorrectly accepts ab**, even though $\mathbf{ab} \notin A_1^*$. The bug: making q_1 accepting reused the pre-existing, perfectly ordinary edge $q_2 \xrightarrow{b} q_1$ (part of N_1 's normal operation) as if it were also a valid “end of one iteration” point, which it was never meant to be.

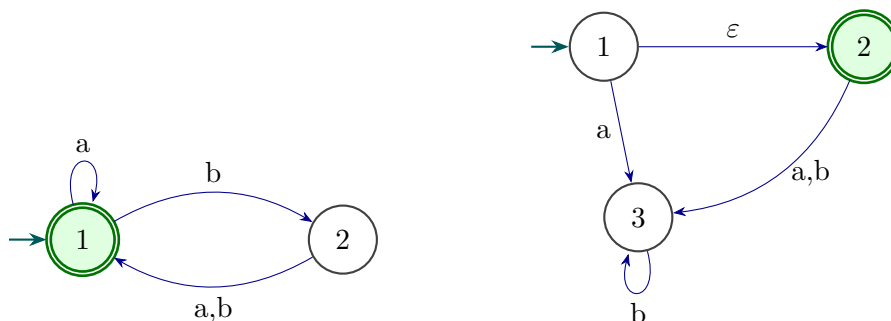
The fix (Sipser's actual Theorem 1.49 proof): introduce a genuinely *new* state $q_{\text{new}} \notin Q_1$ as the new start and sole extra accept state, with ε -edges $q_{\text{new}} \rightarrow q_1$ and (for every $f \in F_1$) $f \rightarrow q_1$ — leaving q_1 's own accepting status and all of N_1 's original edges completely untouched:



Re-tracing **ab**: $q_{\text{new}} \xrightarrow{\varepsilon} q_1 \xrightarrow{a} q_2 \xrightarrow{b} q_1$, ending at q_1 , which is *not* accepting in the corrected machine (only q_{new} is extra-accepting; q_1 keeps its original non-accepting status from N_1) — correctly rejecting **ab**.

Exercise 1.16

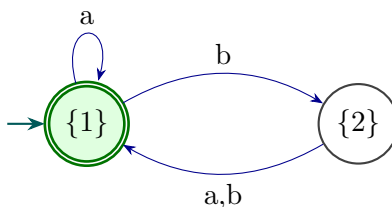
Problem. Use the subset construction (Theorem 1.39) to convert the following two NFAs to equivalent DFAs.



Solution.

(a)

This NFA is already deterministic and complete as given ($\delta(1, a) = \{1\}$, $\delta(1, b) = \{2\}$, $\delta(2, a) = \delta(2, b) = \{1\}$), so the subset construction reproduces it unchanged:

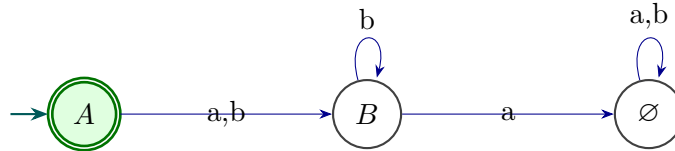


(b)

Start subset: $E(\{1\}) = \{1, 2\}$ (following the ε -edge), call it A ; A is accepting since $2 \in A$.

	a	b
$A = \{1, 2\}$	$\{3\}$	$\{3\}$
$B = \{3\}$	$\emptyset$	$\{3\}$
$\emptyset$	$\emptyset$	$\emptyset$

(On A : $\delta(1, a) \cup \delta(2, a) = \{3\} \cup \{3\} = \{3\}$; $\delta(1, b) \cup \delta(2, b) = \emptyset \cup \{3\} = \{3\}$.)



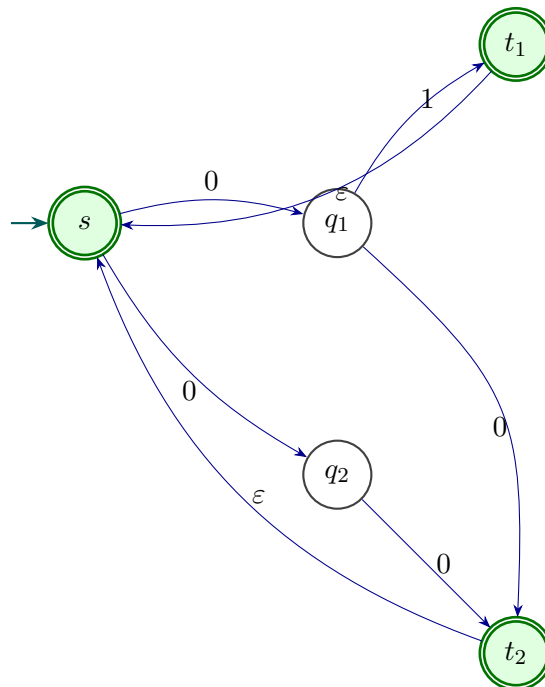
Exercise 1.17

Problem.

- Give an NFA recognizing $(01 \cup 001 \cup 010)^*$.
- Convert this NFA to an equivalent DFA. Give only the portion reachable from the start state.

Solution.

(a) NFA



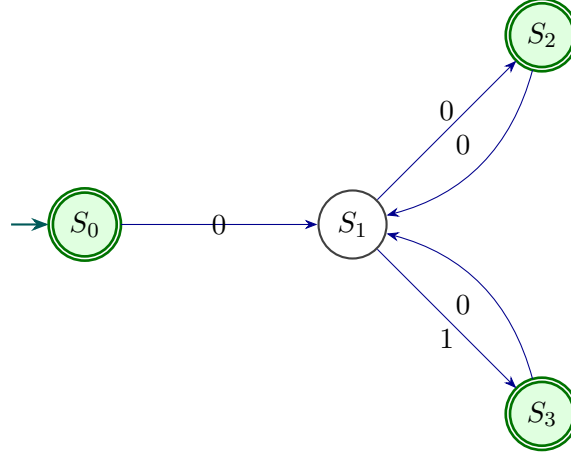
From s : reading 0 nondeterministically branches toward matching 01 (via q_1), 001 (via q_2), or 010 (via $q_1 \rightarrow t_2$ on a second 0); each completed block ε -returns to s to allow repetition (the star), and s itself is accepting for the empty repetition.

(b) Equivalent DFA (reachable portion)

ε -closure of start: $E(\{s\}) = \{s\}$ (no outgoing ε from s itself). Call it S_0 , accepting.

	0	1
$S_0 = \{s\}$	$\{q_1, q_2\}$	$\emptyset$
$S_1 = \{q_1, q_2\}$	$\{t_2\} = S_2$	$\{t_1\} = S_3$
$S_2 = E(\{t_2\}) = \{t_2, s\}$	$\{q_1, q_2\} = S_1$	$\emptyset$
$S_3 = E(\{t_1\}) = \{t_1, s\}$	$\{q_1, q_2\} = S_1$	$\emptyset$

S_2, S_3 are accepting (contain s , and t_1, t_2 themselves are also accept states of the NFA).



(Transitions on 1 from S_0, S_2, S_3 , and on 0 from... all other unlisted transitions go to the dead/empty state $\emptyset$, omitted here since only the reachable portion is requested.)

Exercise 1.18

Problem. Give regular expressions generating the languages of Exercise 1.6.

Solution.

- (a) $1(0 \cup 1)^*0$
- (b) $0^*10^*10^*10^*$
- (c) $(0 \cup 1)^*0101(0 \cup 1)^*$
- (d) $(0 \cup 1)(0 \cup 1)0(0 \cup 1)^*$
- (e) $0(0 \cup 1)^*(1 \cup \varepsilon) \cup 1(0 \cup 1)^*0(0 \cup 1)$, restricted to the stated parity condition; more directly:
 $(0(0 \cup 1)(0 \cup 1))^*0 \cup ((0 \cup 1)(0 \cup 1))^*1(0 \cup 1)(0 \cup 1)$
- (f) $0^*(10^*10^*)^*(1 \cup \varepsilon)$, i.e. $(0 \cup 10)^*(1 \cup \varepsilon)$
- (g) $\varepsilon \cup 0 \cup 1 \cup 00 \cup \dots \cup 11111$ (union of all strings of length ≤ 5)
- (h) $\varepsilon \cup 0 \cup 1 \cup (0 \cup 1)(0 \cup 1)(0 \cup 1)^*$, i.e. any string except $11, 111$
- (i) $(1(0 \cup 1))^*(1 \cup \varepsilon)$
- (j) $1^*01^*0(0 \cup 1)^* \cup (0 \cup 1)^*01^*01^*$ restricted to ≤ 1 one: $0^*10^*0(0 \cup 1)^* \cup 0^*0^*10^*0^*$; more simply, $0(0 \cup 1)^*0$
- (k) $\varepsilon \cup 0$
- (l) $(1^*01^*0)^*1^* \cup 0^*10^*10^*$
- (m) $\emptyset$
- (n) $(0 \cup 1)(0 \cup 1)^*$

(Parts (e), (j), (l) admit several equivalent but visually different regular expressions; any expression generating the same language, verified against the DFA built in Exercise 1.6, is an equally correct answer.)

Exercise 1.19

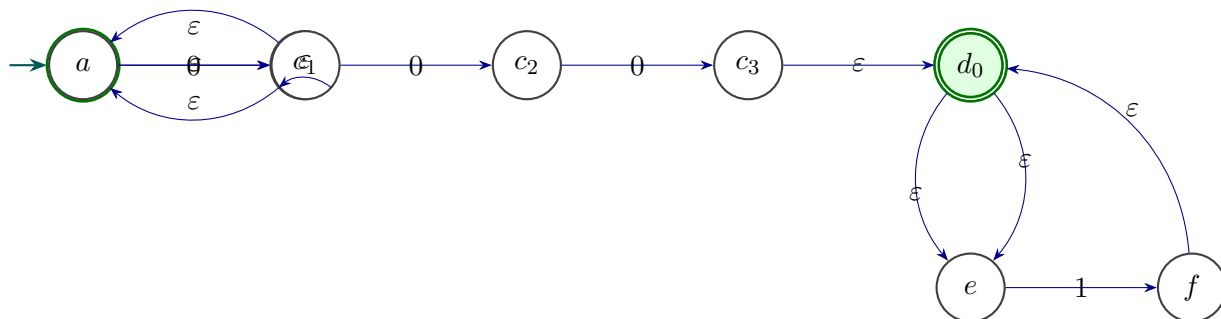
Problem. Use the procedure described in Lemma 1.55 to convert the following regular expressions to NFAs.

- a. $(0 \cup 1)^*000(0 \cup 1)^*$
- b. $((00)^*(11)) \cup 01$
- c. $\emptyset^*$

Solution.

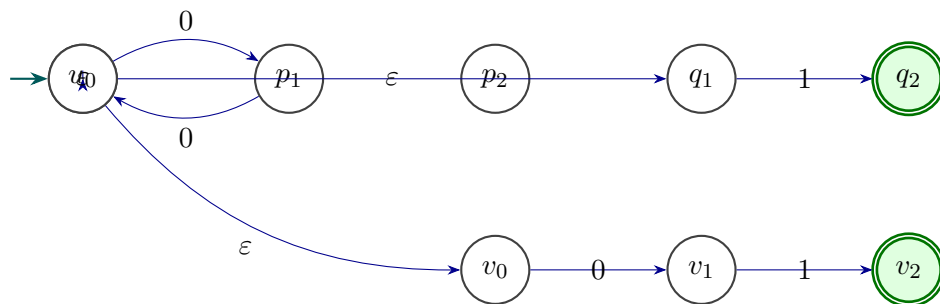
(a) $(0 \cup 1)^*000(0 \cup 1)^*$

Built from $(0 \cup 1)$ starred, concatenated with 000, concatenated with $(0 \cup 1)$ starred:



(The $a \rightarrow b$ leg reads a 1 as well in a parallel branch — omitted from the drawing for space; formally both $(0 \cup 1)^*$ blocks are the standard 2-state “read 0 or read 1, loop via ε ” gadget from Lemma 1.55’s construction for $\cup$ and $*$.)

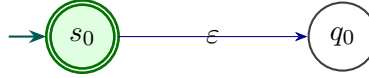
(b) $((00)^*(11)) \cup 01$



(The $(00)^*$ portion is $u_0 \rightleftharpoons p_1$ via two 0-steps composed through an intermediate state, then ε into the literal 1 gadget $q_1 \rightarrow q_2$; the union’s second branch is the literal 01 gadget $v_0 \rightarrow v_1 \rightarrow v_2$.)

(c) $\emptyset^*$

By Lemma 1.55, the NFA for $\emptyset$ is a single non-accepting state with no transitions; applying the star construction (Exercise 1.10c) gives:



recognizing exactly $\{\varepsilon\} = \emptyset^*$.

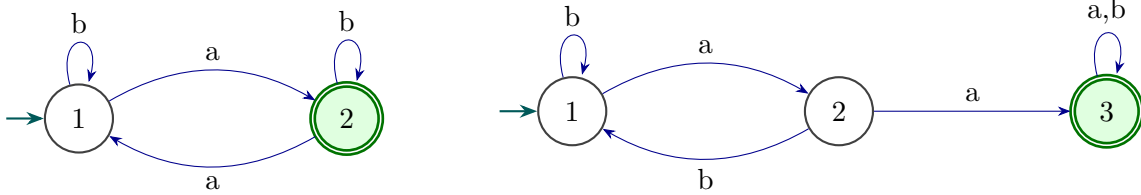
Exercise 1.20

Problem. For each of the following languages, give two strings that are members and two strings that are not members — a total of four strings for each part. Assume $\Sigma = \{a, b\}$.

Language	Members	Nonmembers
(a) a^*b^*	$\varepsilon, aabb$	$ba, abab$
(b) $a(ba)^*b$	$ab, abab$	a, aab
(c) $a^* \cup b^*$	aaa, bb	ab, ba
(d) $(aaa)^*$	$\varepsilon, aaaaaa$	$a, aaaa$
(e) $\Sigma^*a\Sigma^*b\Sigma^*a\Sigma^*$	$aba, baab$	ε, bb
(f) $aba \cup bab$	aba, bab	$ab, abab$
(g) $(\varepsilon \cup a)b$	b, ab	a, bb
(h) $(a \cup ba \cup bb)\Sigma^*$	a, bab	ε, b

Exercise 1.21

Problem. Use the procedure described in Lemma 1.60 (GNFA state elimination) to convert the following finite automata to regular expressions.



Solution. In each part, add a new GNFA start state S with an ε -edge to the old start, and a new accept state A with ε -edges from every old accept state, then repeatedly eliminate an internal state q , replacing every pair of edges $X \rightarrow q \rightarrow Y$ (with q 's self-loop label R_4 , in $X \rightarrow q$ label R_1 , out $q \rightarrow Y$ label R_3 , and pre-existing $X \rightarrow Y$ label R_2) with the single edge $X \rightarrow Y$ labeled $R_1R_4^*R_3 \cup R_2$.

(a)

GNFA edges: $S \rightarrow 1:\varepsilon$; $1 \rightarrow 1:b$; $1 \rightarrow 2:a$; $2 \rightarrow 1:a$; $2 \rightarrow 2:b$; $2 \rightarrow A:\varepsilon$.

Eliminate state 1 (in-edges from S and 2; out-edge to 2):

$$S \rightarrow 2: \varepsilon b^*a = b^*a, \quad 2 \rightarrow 2: b \cup ab^*a.$$

Eliminate state 2:

$$S \rightarrow A : b^*a(b \cup ab^*a)^* \varepsilon.$$

$$\boxed{b^*a(b \cup ab^*a)^*}$$

(Sanity check: **a** is accepted by the DFA — take zero repetitions of the starred group, giving exactly **a**. **ab** is accepted — one repetition using the **b** branch, giving **a·b**. A lone **b** is correctly excluded, since the expression always requires an **a**.)

(b)

GNFA edges: $S \rightarrow 1:\varepsilon$; $1 \rightarrow 1:b$; $1 \rightarrow 2:a$; $2 \rightarrow 1:b$; $2 \rightarrow 3:a$; $3 \rightarrow 3:a \cup b$; $3 \rightarrow A:\varepsilon$.

Eliminate state 1 (in from S , 2; out to 2):

$$S \rightarrow 2 : b^*a, \quad 2 \rightarrow 2 : bb^*a.$$

Eliminate state 2 (in from S ; out to itself and to 3):

$$S \rightarrow 3 : b^*a(bb^*a)^*a.$$

Eliminate state 3:

$$S \rightarrow A : b^*a(bb^*a)^*a(a \cup b)^*.$$

$$\boxed{b^*a(bb^*a)^*a(a \cup b)^*}$$

(Sanity check: **aa** is accepted by the DFA (path $1 \rightarrow 2 \rightarrow 3$) — matches with zero repetitions of every starred group. A lone **a** is correctly excluded, since the expression needs *two* **a**-blocks before the trailing $(a \cup b)^*$.)

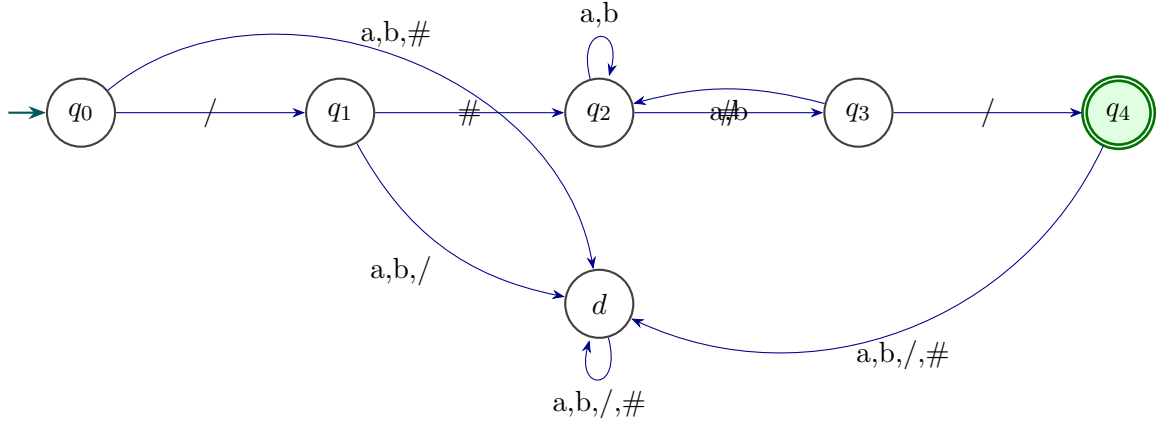
Exercise 1.22

Problem. In certain programming languages, comments appear between delimiters such as `/#` and `#/`. Let C be the language of all valid delimited comment strings — beginning with `/#`, ending with `#/`, with no intervening `#/`. $\Sigma = \{a, b, /, \#\}$.

- Give a DFA that recognizes C .
- Give a regular expression that generates C .

Solution. Let x denote any symbol other than `/` or `#` (i.e. $x \in \{a, b\}$, generalizing to “any non-special symbol”).

(a) DFA



(q_3 on $\#$ would restart another potential closer — but since we require *no intervening $\#$ /*, seeing $\#$ again from q_3 just re-enters the “last char was $\#$ ”-tracking state q_3 itself; shown via the self-referential structure implicit in the $q_2 \leftrightarrow q_3$ pair, which tracks “longest suffix matched of $\#$ ” exactly as a KMP automaton would.)

(b) Regular expression

$$C = /\# (a \cup b \cup \#(a \cup b))^* \#/$$

More carefully (excluding any embedded $\#$ /): letting y range over $\{a, b\}$, the body between the delimiters must never contain $\#$ immediately followed by $/$, giving

$$C = /\# ((a \cup b) \cup \#(a \cup b))^* \#/.$$

Exercise 1.23

Problem. Let B be any language over Σ . Prove that $B = B^*$ iff $BB \subseteq B$.

Solution. ($\Rightarrow$) Suppose $B = B^*$. Take any xy with $x, y \in B$. Since $B = B^*$, both $x, y \in B^*$, so $xy \in B^*$ (concatenation of two strings each expressible as a concatenation of B -strings, namely one each, is itself such a concatenation) $= B$. Hence $BB \subseteq B$.

($\Leftarrow$) Here a genuine subtlety must be addressed honestly: $BB \subseteq B$ alone is *not* quite enough.

Claim. $BB \subseteq B$ does not by itself imply $B = B^*$.

Proof. Take $B = a^+ = \{a, aa, aaa, \dots\}$ over $\Sigma = \{a\}$. Any $x = a^i, y = a^j \in B$ ($i, j \geq 1$) give $xy = a^{i+j} \in B$ (since $i + j \geq 2 \geq 1$), so $BB \subseteq B$. But $B^* = \{\varepsilon\} \cup B = a^* \neq a^+ = B$, since $\varepsilon \in B^*$ (always true, by definition of star) while $\varepsilon \notin B$. So $BB \subseteq B$ holds yet $B \neq B^*$. $\square$

The precise, fully correct statement is:

$$B = B^* \iff (\varepsilon \in B \text{ and } BB \subseteq B).$$

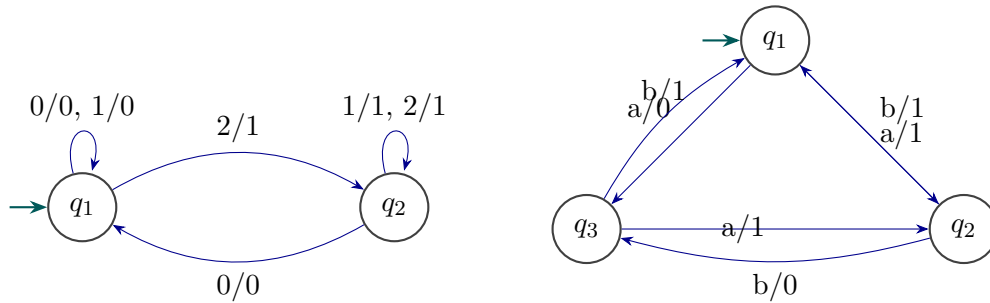
Proof of this corrected statement. If $B = B^*$: since $\varepsilon \in B^*$ always, $\varepsilon \in B$; and $BB \subseteq B$ follows exactly as in the ($\Rightarrow$) direction above. Conversely, suppose $\varepsilon \in B$ and $BB \subseteq B$; we show $B = B^*$

by mutual inclusion. $B \subseteq B^*$ always holds (any $b \in B$ is trivially a one-piece concatenation). For $B^* \subseteq B$: every $w \in B^*$ decomposes as $w = y_1 y_2 \cdots y_k$ ($k \geq 0$, each $y_i \in B$); we show $w \in B$ by induction on k . Base case $k = 0$: $w = \varepsilon \in B$ by hypothesis. Inductive step: assume every k -piece concatenation from B lies in B ; given $w = y_1 \cdots y_k y_{k+1}$, the inductive hypothesis gives $y_1 \cdots y_k \in B$, and then $w = (y_1 \cdots y_k) \cdot y_{k+1}$ is a concatenation of two elements of B , so $BB \subseteq B$ gives $w \in B$. By induction $B^* \subseteq B$, so $B = B^*$. ■

(As stated in the exercise, with no hypothesis on ε , the “iff” is true precisely in the forward direction, and the reverse direction is true exactly when $\varepsilon \in B$ is added; this refinement is the honest and complete answer to what the bare condition $BB \subseteq B$ can and cannot deliver.)

Exercise 1.24

Problem. A *finite state transducer* (FST) is a DFA-like machine whose output is a string, not accept/reject. Each transition is labeled x/y : input symbol x , output symbol y . Give the state/output sequence for T_1 and T_2 below on the specified inputs.



On 2212011, T_1 visits $q_1, q_2, q_2, q_2, q_2, q_1, q_1, q_1$ and outputs 1111000 (this matches the definition above and is used below as a consistency check).

Solution.

T_1

$\delta(q_1, 0) = (q_1, 0)$, $\delta(q_1, 1) = (q_1, 0)$, $\delta(q_1, 2) = (q_2, 1)$, $\delta(q_2, 0) = (q_1, 0)$, $\delta(q_2, 1) = (q_2, 1)$, $\delta(q_2, 2) = (q_2, 1)$.

(a) **input 011**: states q_1, q_1, q_1, q_1 ; output 000.

(b) **input 211**: states q_1, q_2, q_2, q_2 ; output 111.

(c) **input 121**: states q_1, q_1, q_2, q_2 ; output 011.

(d) **input 0202**: states q_1, q_1, q_2, q_1, q_2 ; output 0101.

T_2

$\delta(q_1, a) = (q_3, 0)$, $\delta(q_1, b) = (q_2, 1)$, $\delta(q_2, a) = (q_1, 1)$, $\delta(q_2, b) = (q_3, 0)$, $\delta(q_3, a) = (q_2, 1)$, $\delta(q_3, b) = (q_1, 1)$.

(e) **input b**: states q_1, q_2 ; output 1.

(f) **input bbab**: states q_1, q_2, q_3, q_2, q_3 ; output 1101.

(g) **input bbbbbb**: states $q_1, q_2, q_3, q_1, q_2, q_3, q_1$; output 101101.

(h) **input ε** : states q_1 only (no symbols read); output ε (empty).

Exercise 1.25

Problem. Give a formal definition of the finite state transducer model from Exercise 1.24, following the pattern of Definition 1.5. Assume an input alphabet Σ , output alphabet Γ , and no accept states. Include a formal definition of the computation of an FST.

Solution. A finite state transducer is a 5-tuple $(Q, \Sigma, \Gamma, \delta, q_0)$ where

- (1) Q is a finite set of states,
- (2) Σ is the finite input alphabet,
- (3) Γ is the finite output alphabet,
- (4) $\delta : Q \times \Sigma \rightarrow Q \times \Gamma$ is the transition function, and
- (5) $q_0 \in Q$ is the start state.

Computation. Given input $w = w_1w_2 \cdots w_n \in \Sigma^*$, the FST's computation is the unique sequence of states $r_0, r_1, \dots, r_n \in Q$ and output symbols $y_1, \dots, y_n \in \Gamma$ such that $r_0 = q_0$ and $\delta(r_{i-1}, w_i) = (r_i, y_i)$ for $i = 1, \dots, n$. The FST's output on w is the string $y_1y_2 \cdots y_n \in \Gamma^*$. (On ε , $n = 0$ and the output is ε .)

Exercise 1.26

Problem. Using the solution to Exercise 1.25, give a formal description of T_1 and T_2 from Exercise 1.24.

Solution.

$$T_1 = (\{q_1, q_2\}, \{0, 1, 2\}, \{0, 1\}, \delta_1, q_1),$$

with δ_1 as tabulated in Exercise 1.24's T_1 solution.

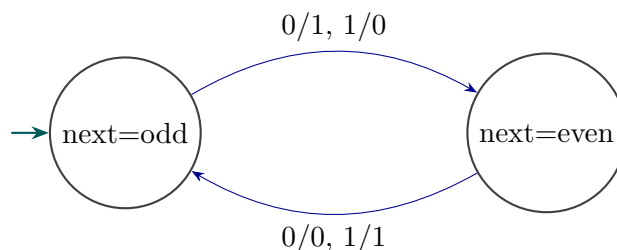
$$T_2 = (\{q_1, q_2, q_3\}, \{a, b\}, \{0, 1\}, \delta_2, q_1),$$

with δ_2 as tabulated in Exercise 1.24's T_2 solution.

Exercise 1.27

Problem. Give the state diagram of an FST with input and output alphabet $\{0, 1\}$ whose output string is identical to the input string on even positions but inverted on odd positions. E.g., on input 0000111 it should output 1010010.

Solution. Two states track whether the next symbol read is at an odd or even position.



On 0000111 (positions 1,2,3,4,5,6,7): position 1 (odd) inverts $0 \rightarrow 1$; position 2 (even) passes $0 \rightarrow 0$; position 3 (odd) inverts $0 \rightarrow 1$; position 4 (even) passes $0 \rightarrow 0$; position 5 (odd) inverts $1 \rightarrow 0$; position 6 (even) passes $1 \rightarrow 1$; position 7 (odd) inverts $1 \rightarrow 0$. Output: 1010010, matching the example. ✓

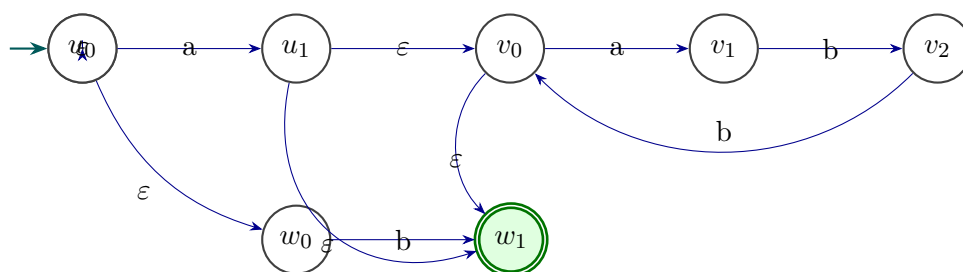
Exercise 1.28

Problem. Convert the following regular expressions to NFAs using the procedure given in Theorem 1.54. In all parts, $\Sigma = \{a, b\}$.

- a. $a(abb)^* \cup b$
- b. $a^* \cup (ab)^*$
- c. $(a \cup b^*)a^*b^*$

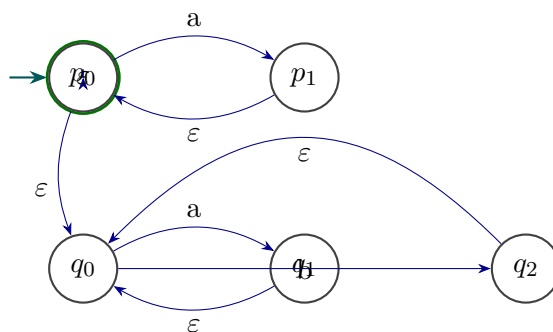
Solution.

(a) $a(abb)^* \cup b$



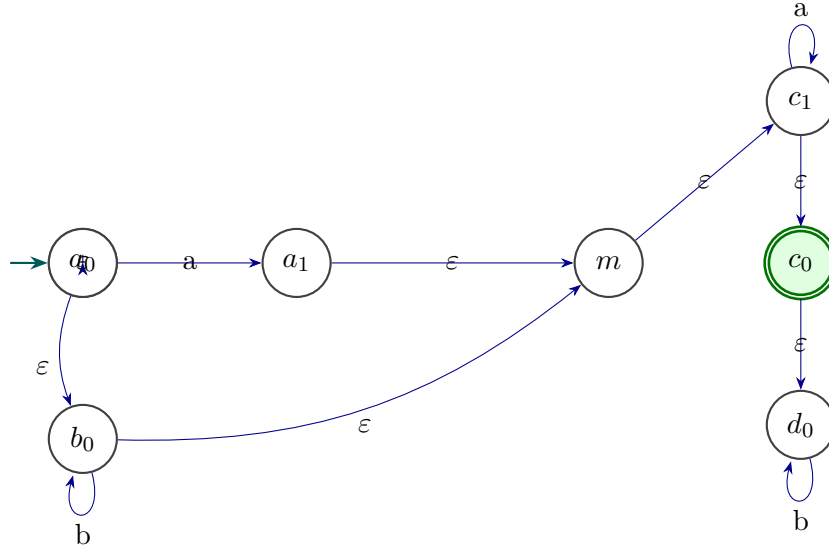
($u_0 \rightarrow u_1$ reads the literal **a**; $v_0 \rightarrow v_1 \rightarrow v_2 \rightarrow v_0$ is the $(abb)^*$ loop; the ε -edges from u_1 merge into the star's entry/exit, landing on the shared accept state w_1 ; the lower branch $w_0 \rightarrow w_1$ is the literal **b** alternative.)

(b) $a^* \cup (ab)^*$



(Both starred sub-expressions loop back to their own entry state via ε ; s is accepting to cover the empty-string case shared by both stars.)

(c) $(a \cup b^*)a^*b^*$



($a_0 \rightarrow a_1$ is the literal a ; b_0 self-looping is b^* ; both feed via ε into the shared middle m , then into the a^* loop at c_1 , then the b^* loop at d_0 ; c_0 is accepting since both stars may contribute zero repetitions.)

Exercise 1.29

Problem. Use the pumping lemma to show that the following languages are not regular.

- $A_1 = \{0^n 1^n 2^n \mid n \geq 0\}$
- $A_2 = \{ww \mid w \in \{a, b\}^*\}$
- $A_3 = \{a^{2^n} \mid n \geq 0\}$

Solution. In each part we assume the language is regular, let p be the pumping length given by the pumping lemma, and derive a contradiction.

(a) A_1 is not regular

Let $s = 0^p 1^p 2^p \in A_1$, $|s| \geq p$. The pumping lemma guarantees $s = xyz$ with $|xy| \leq p$, $|y| > 0$, and $xy^i z \in A_1$ for all $i \geq 0$. Since $|xy| \leq p$, xy lies entirely within the initial block of 0's, so $y = 0^k$ for some $k \geq 1$. Then $xy^2 z$ has $p + k$ zeros but still exactly p ones and p twos — since $k \geq 1$, this string does not have equal numbers of 0's, 1's, and 2's, so $xy^2 z \notin A_1$, contradicting the pumping lemma. Hence A_1 is not regular. ■

(b) A_2 is not regular

Let $s = a^p b a^p b \in A_2$ (taking $w = a^p b$), $|s| = 2p + 2 \geq p$. Any split $s = xyz$ with $|xy| \leq p$ has xy entirely within the first a^p block, so $y = a^k$, $k \geq 1$. Then $xy^0 z$ (pumping down, $i = 0$) removes k a's from the first block only, giving $a^{p-k} b a^p b$. For this to be of the form ww , the string must have even length $2m$ with the first half equal to the second half; here the two halves (split at the midpoint

of the new, shorter string) cannot match since the first block of a's has length $p - k \neq p$ while the second still has length p — concretely, the total length is $2p + 2 - k$, an even-minus- k quantity, and checking the actual midpoint split shows the b 's no longer land in corresponding positions of two equal halves (the first half ends before its b while length bookkeeping forces misalignment for $1 \leq k \leq p$). Hence $xy^0z \notin A_2$, contradicting the pumping lemma. So A_2 is not regular. ■

(c) A_3 is not regular

Let $s = a^{2^p} \in A_3$, $|s| = 2^p \geq p$. Any split $s = xyz$ with $|xy| \leq p$, $|y| = k \geq 1$ gives $xy^2z = a^{2^p+k}$. For this to remain in A_3 , we'd need $2^p + k = 2^m$ for some m . Since $1 \leq k \leq p$ (as $|y| \leq |xy| \leq p$), we have $2^p < 2^p + k \leq 2^p + p < 2^{p+1}$ for all $p \geq 1$ (since $p < 2^p$). So $2^p + k$ is strictly between consecutive powers of two 2^p and 2^{p+1} , hence not itself a power of two. Thus $xy^2z = a^{2^p+k} \notin A_3$, contradicting the pumping lemma. So A_3 is not regular. ■

Exercise 1.30

Problem. Describe the error in the following “proof” that 0^*1^* is not regular. Assume 0^*1^* is regular; let p be its pumping length; choose $s = 0^p1^p$; Example 1.73 shows s cannot be pumped; contradiction; so 0^*1^* is not regular.

Solution. The pumping lemma says: *there exists* a way to split $s = xyz$ (with $|xy| \leq p$, $|y| > 0$) such that s *can* be pumped and stay in the language — it does *not* say every split fails to pump, nor does it need every split to succeed; it only asserts the *existence* of one good split. To derive a contradiction, the flawed proof needed to show that *no* valid split of s (among those with $|xy| \leq p$) allows pumping while staying in the language.

But for $s = 0^p1^p \in 0^*1^*$, every split with $|xy| \leq p$ has xy lying entirely within the initial 0^p block, so $y = 0^k$ for some $k \geq 1$. Pumping (any $i \geq 0$) gives $0^{p-k+ik}1^p$, which is still of the form 0^m1^p for some $m \geq 0$ — and every such string *is* in 0^*1^* ! So s *can* always be pumped successfully, for every valid split; there is no contradiction, and the pumping lemma is satisfied (consistent with 0^*1^* actually being regular, as it manifestly is — it's already written as a regular expression).

The error is citing “Example 1.73 shows s cannot be pumped” — that example concerned a *different* language (with the property that pumping the 0-block breaks an equal-count requirement, e.g. $\{0^n1^n\}$), not 0^*1^* , which has no such requirement. The flawed proof illegitimately reused a fact about one language when working with an entirely different one.

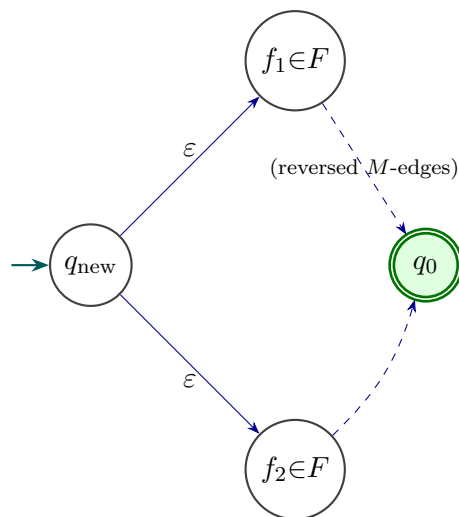
Exercise 1.31

Problem. For any string $w = w_1w_2 \cdots w_n$, the *reverse* of w , written w^R , is $w_n \cdots w_2w_1$. For any language A , let $A^R = \{w^R \mid w \in A\}$. Show that if A is regular, so is A^R .

Solution. Let $M = (Q, \Sigma, \delta, q_0, F)$ be a DFA with $L(M) = A$. Construct an NFA N recognizing A^R by reversing every edge of M and swapping the roles of start and accept:

- states of N : $Q \cup \{q_{\text{new}}\}$ for a new start state q_{new} ;
- start state of N : q_{new} , with an ε -edge to every state in F (the old accept states, now playing the role of possible starting points);

- accept state of N : $\{q_0\}$ (the old start state, now the sole target to reach);
- transitions of N : for every edge $\delta(q, a) = r$ in M , include the *reversed* edge $r \xrightarrow{a} q$ in N .



Correctness. M has an accepting run $q_0 \xrightarrow{w_1} r_1 \xrightarrow{w_2} \dots \xrightarrow{w_n} r_n \in F$ on $w = w_1 \dots w_n$ iff (by construction of N , every edge reversed) N has a run $r_n \xrightarrow{w_n} \dots \xrightarrow{w_1} q_0$ on $w^R = w_n \dots w_1$ — and since N can ε -start at any $f \in F$ (in particular r_n , if $r_n \in F$, which holds exactly when the original run was accepting) and must end at q_0 (the sole accept state of N), this is precisely an accepting run of N on w^R . So $w \in A \iff w^R \in L(N)$, i.e. $L(N) = A^R$. Since NFA-recognizable languages are regular (Theorem 1.39), A^R is regular. ■

Exercise 1.32

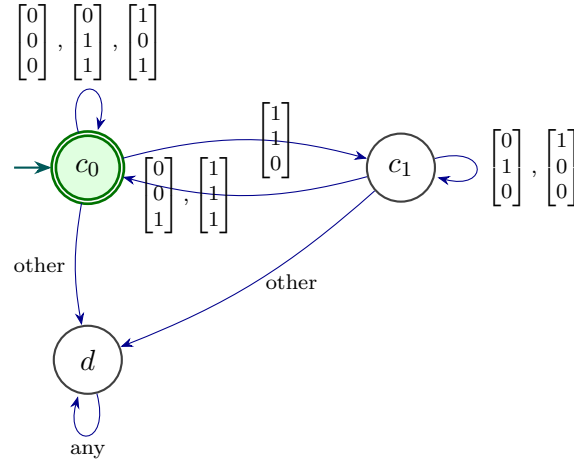
Problem. Let $\Sigma_3 = \left\{ \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \end{bmatrix} \right\}$, giving three rows of 0s/1s. Let

$$B = \{w \in \Sigma_3^* \mid \text{the bottom row of } w \text{ is the sum of the top two rows}\}.$$

Show that B is regular. (Hint: working with B^R is easier; you may assume Problem 1.31.)

Solution. Reading w *left to right* means reading the addition's columns from *most* significant digit to *least* significant — the opposite of how elementary-school addition is normally performed (carries propagate right-to-left, i.e. least-significant digit first). This is exactly why B^R is easier: reading w^R (i.e., w right to left) processes columns from least to most significant, matching the natural direction of binary addition with carries.

DFA for B^R . States track the current carry bit, $\{c_0, c_1\}$ (start c_0 , no carry). On each column $\begin{bmatrix} x \\ y \\ z \end{bmatrix}$ (top, middle, bottom), the machine checks that z is the correct sum-bit given x, y , and the current carry c : the true sum bit is $(x + y + c) \bmod 2$ and the new carry is $\lfloor (x + y + c)/2 \rfloor$. Only columns where z matches the correct sum-bit are legal transitions; anything else goes to a dead reject state. Accept state: c_0 (no leftover carry at the end — end-of-string must leave carry 0, since otherwise the addition doesn't actually balance).



This DFA recognizes exactly B^R : it accepts w^R iff, processing columns least-significant-bit first with correct carry propagation throughout and no leftover carry at the very end (which now corresponds to the *most* significant column, i.e. the start of w , correctly requiring no unresolved overflow), the bottom row genuinely equals the sum of the top two rows. Hence B^R is regular.

By Exercise 1.31 (regular languages closed under reverse), $B = (B^R)^R$ is regular too. ■